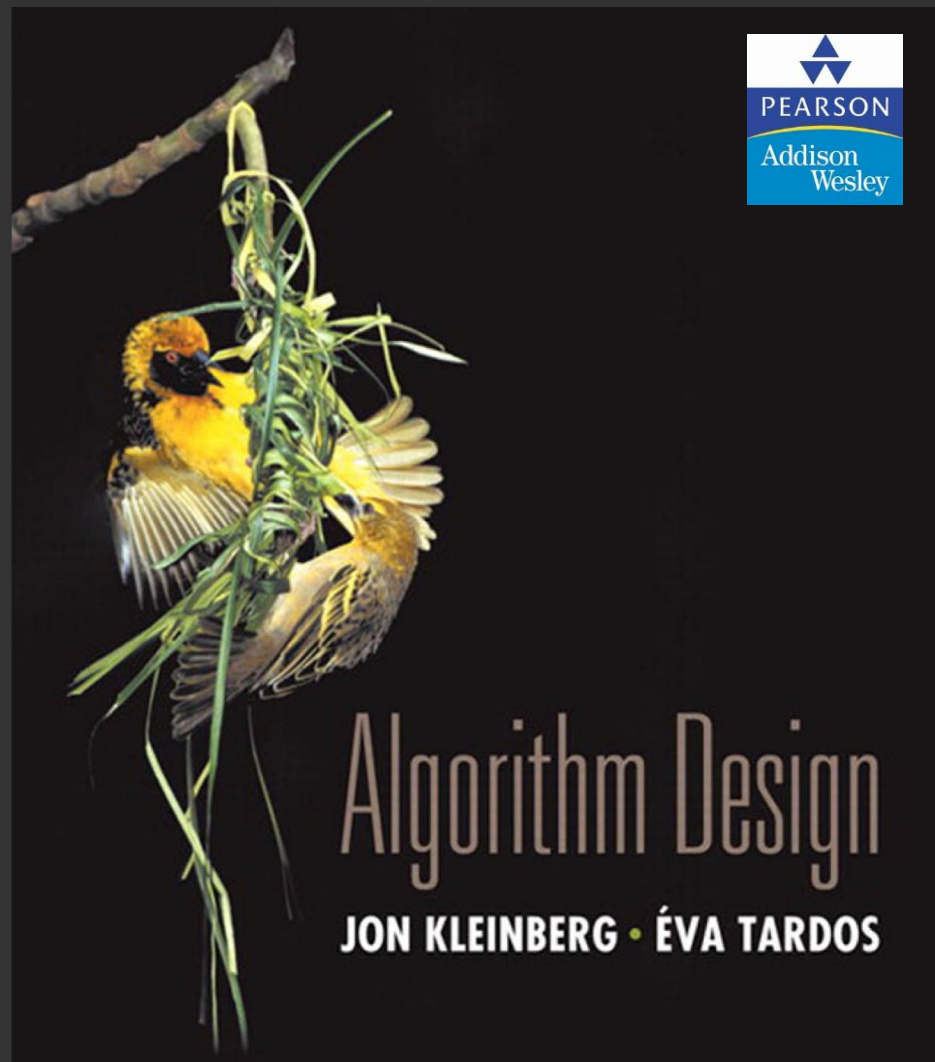




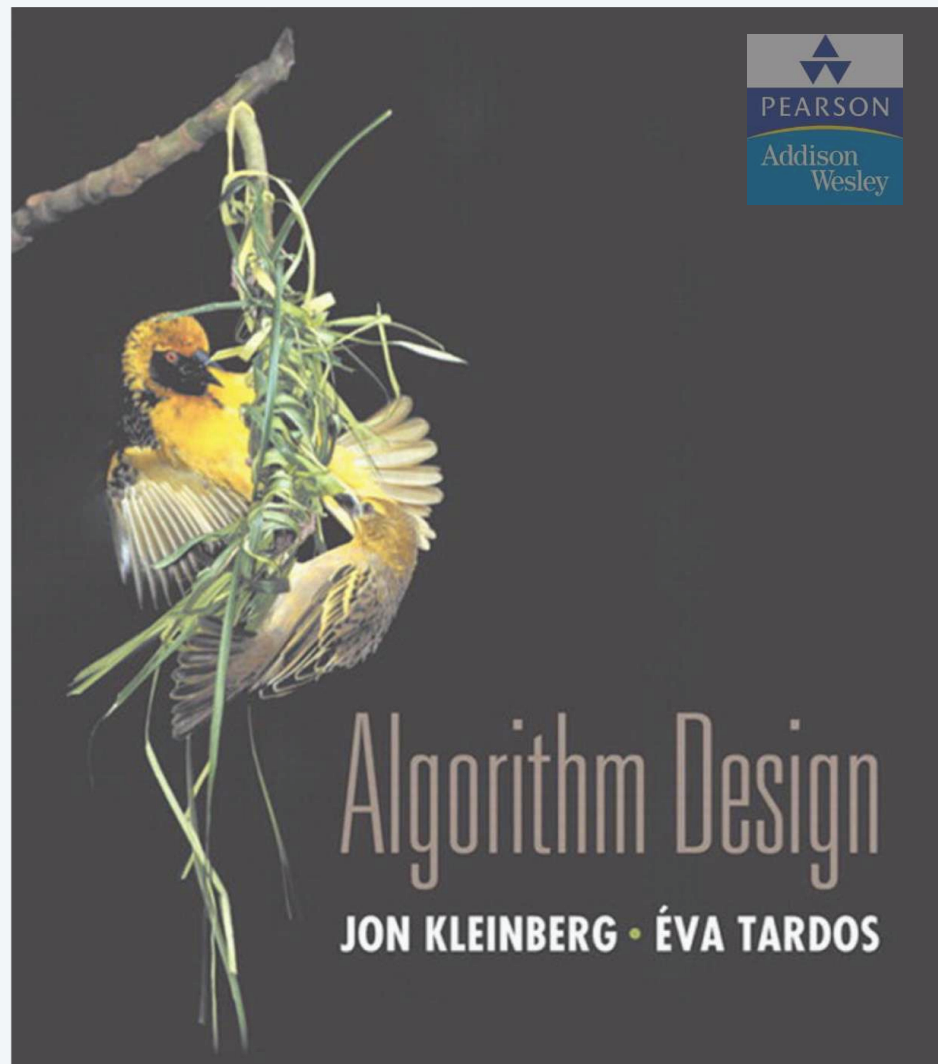
7. NETWORK FLOW I

- *max-flow and min-cut problems*
- *Ford–Fulkerson algorithm*
- *max-flow min-cut theorem*
- *choosing good augmenting paths*

Lecture slides by Kevin Wayne

Copyright © 2005 Pearson-Addison Wesley

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>



SECTION 7.1

7. NETWORK FLOW I

- *max-flow* and *min-cut* problems
- *Ford–Fulkerson algorithm*
- *max-flow min-cut theorem*
- *choosing good augmenting paths*

Flow network

A **flow network** is a tuple $G = (V, E, s, t, c)$.

- Digraph (V, E) with source $s \in V$ and sink $t \in V$.
- Capacity $c(e) \geq 0$ for each $e \in E$.

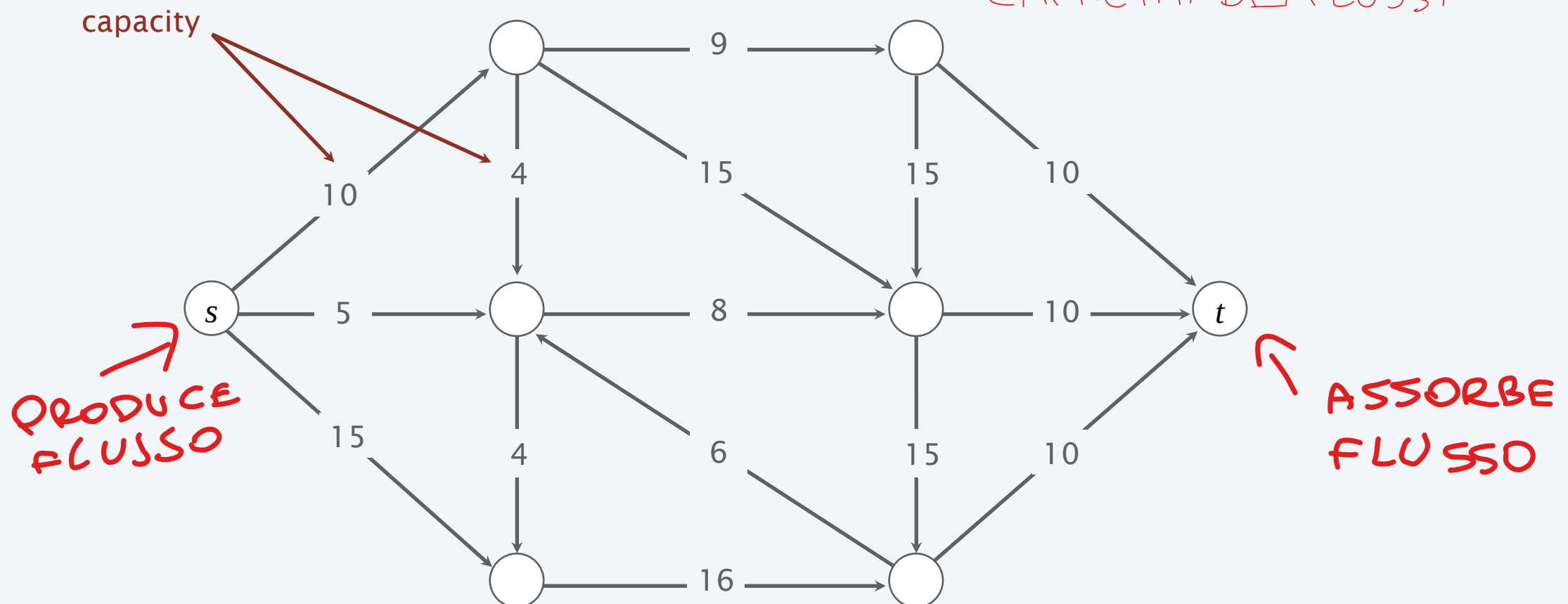
IPESI SIDIKONO CAPACITA'

ARCO

assume all nodes are reachable from s

Intuition. Material flowing through a transportation network; material originates at source and is sent to sink.

*SPOSTARE UN PO' DI ROBA DALLA SORGENTE
AL POZZO → IL TUTTO E' LIMITATO DALLE
CAPACITA' DEI FLUSSI*



Minimum-cut problem

Def. An *st-cut* (cut) is a partition (A, B) of the nodes with $s \in A$ and $t \in B$.

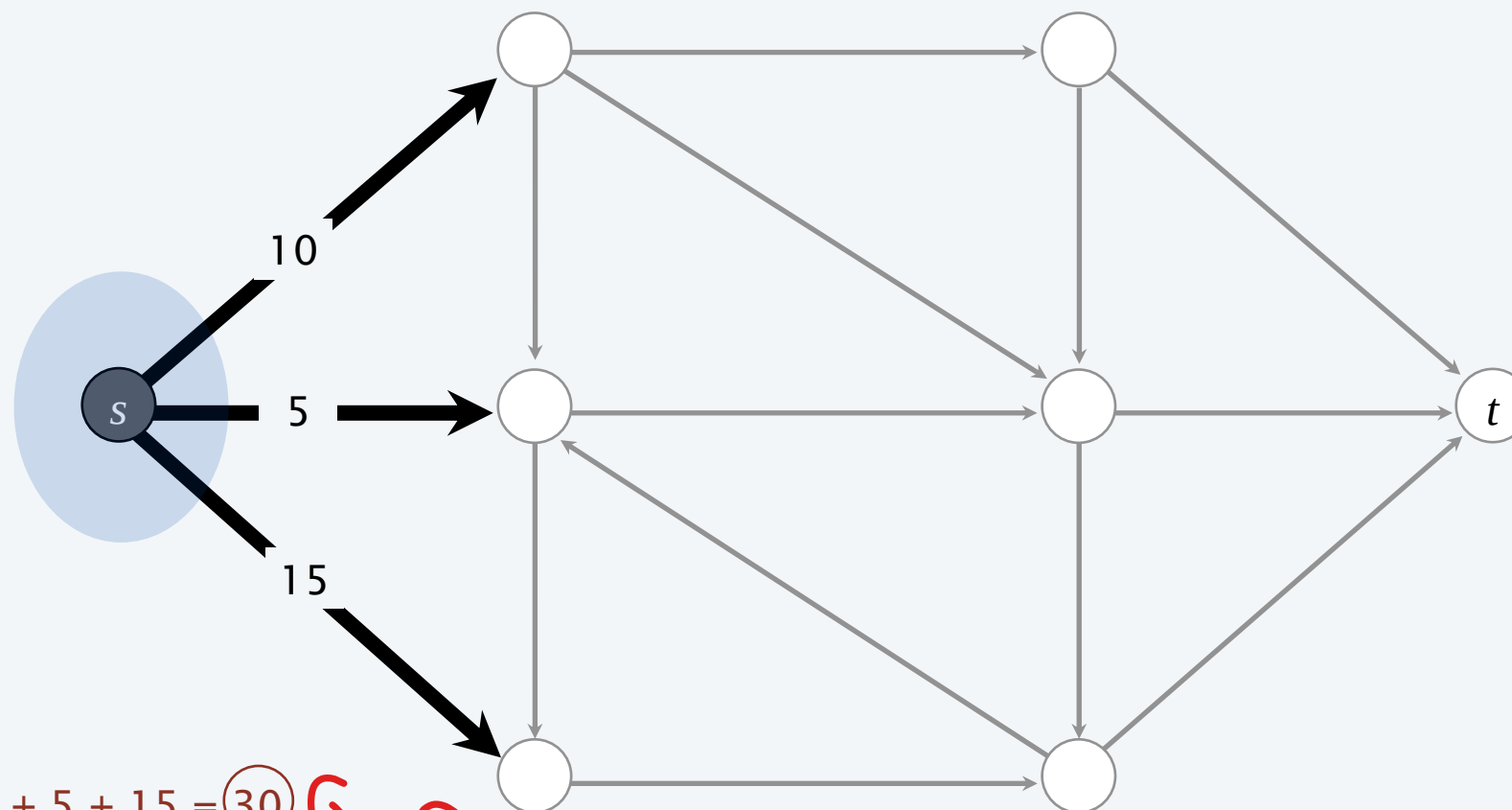
Def. Its *capacity* is the sum of the capacities of the edges from A to B .

$$\text{CAPACITA'}(A, B) \Rightarrow \text{cap}(A, B) = \sum_{e \text{ out of } A} c(e)$$

TUTTI GLI ARCHI CHE LASCIANO A

ST-CUT $\rightarrow$ PARTIZIONE DEI NODI IN PIU' INSIEMI TIPO A/B

DIFF TRA CUT EST-CUT?



capacity = $10 + 5 + 15 = 30$ $\leftarrow$ ES EN 101

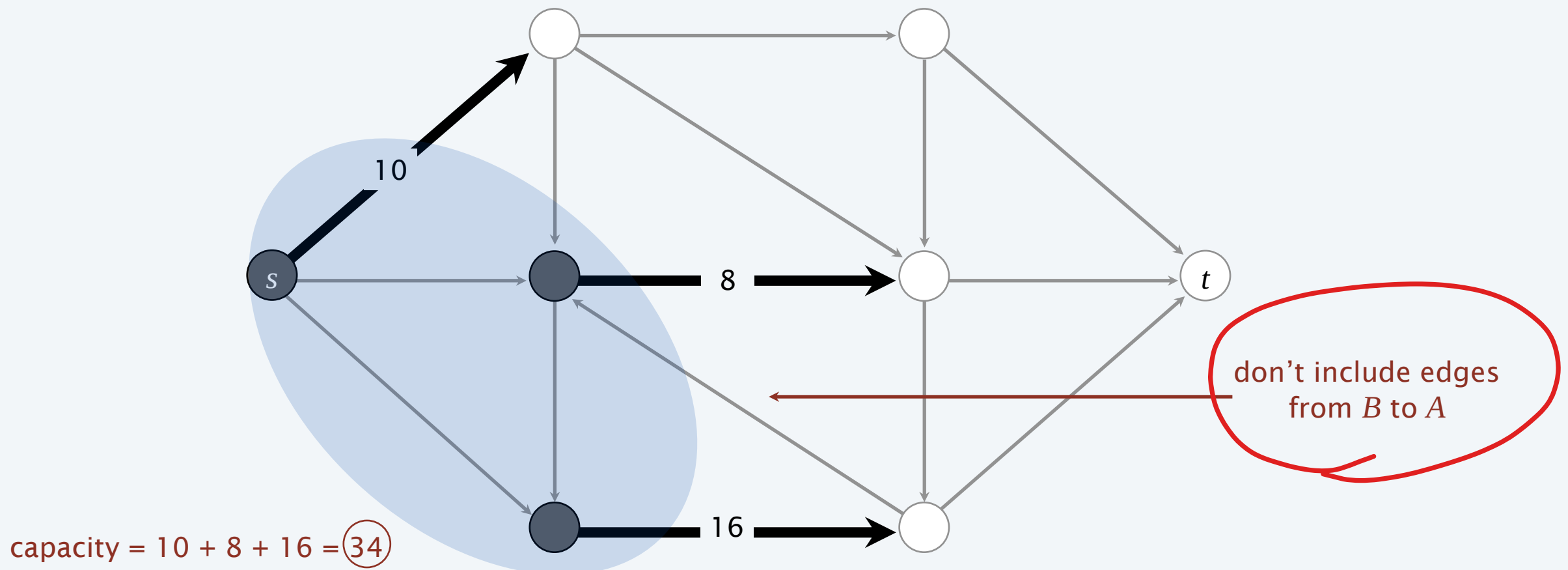
ES EN 102

Minimum-cut problem

Def. An *st-cut (cut)* is a partition (A, B) of the nodes with $s \in A$ and $t \in B$.

Def. Its *capacity* is the sum of the capacities of the edges from A to B .

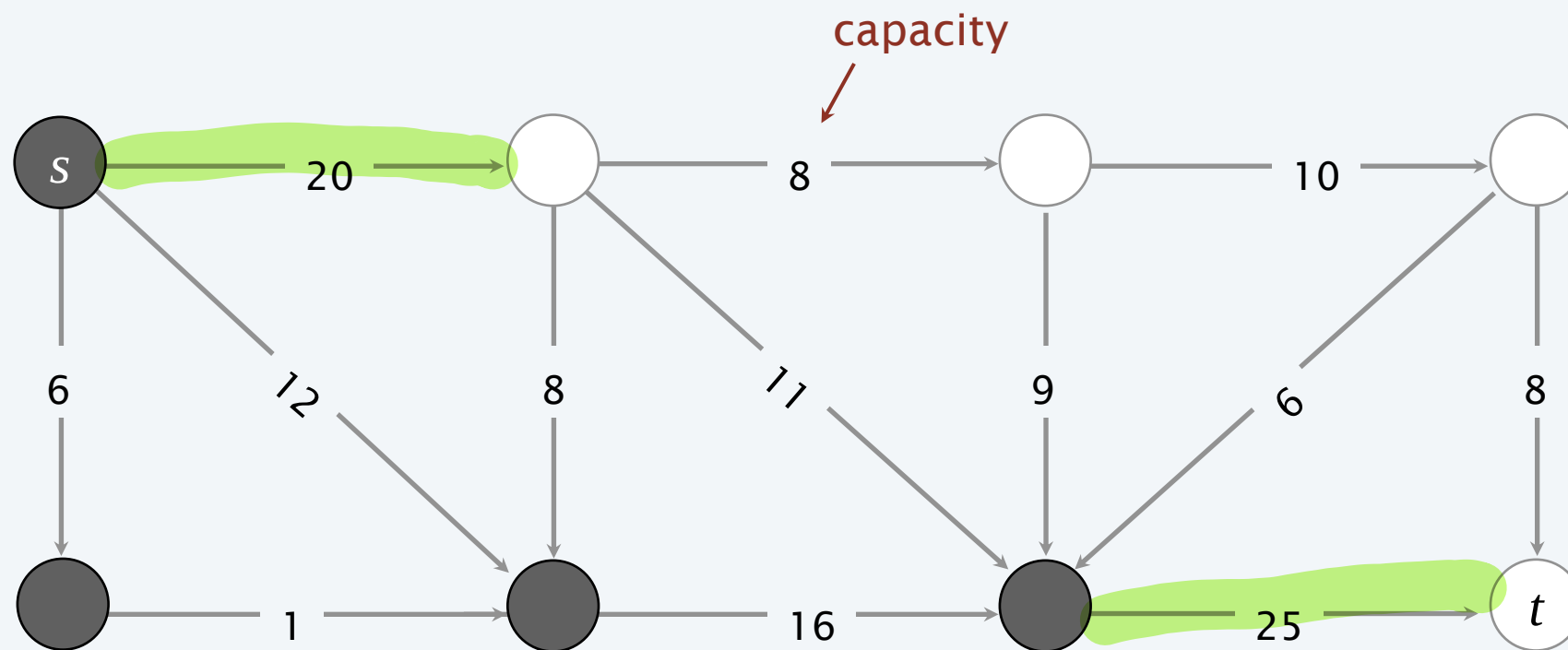
$$\text{cap}(A, B) = \sum_{e \text{ out of } A} c(e)$$



One more example

Capacity of the given st -cut: $20+25=45$

PK PRENDI SOLO
E' UN ESEMPIO CHE PRENDE DUE
ARCHI CHE PORTANO A t , SENZA
CONTARE PASSAGGI INTERMEDI



Minimum-cut problem

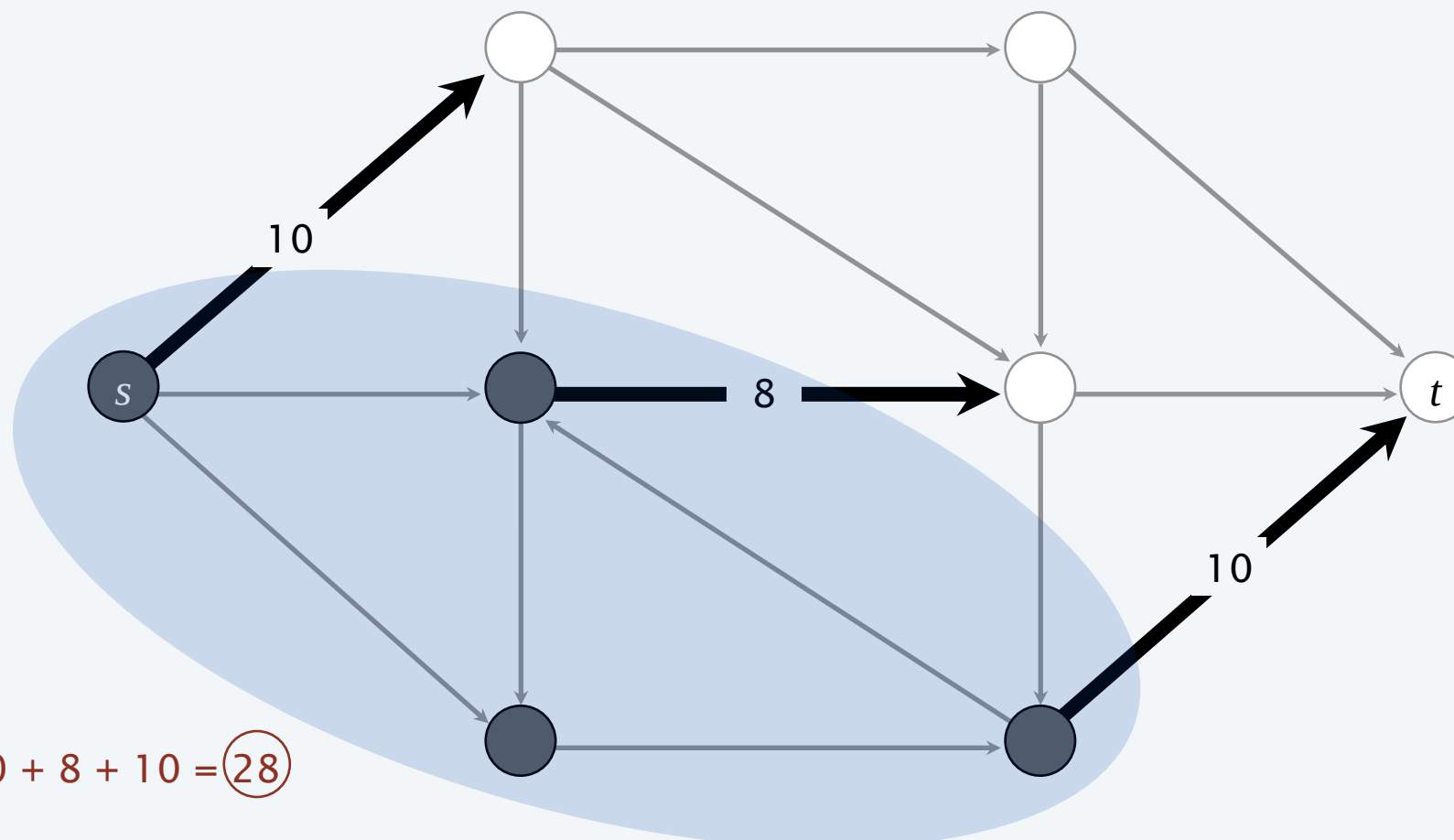
Def. An *st-cut (cut)* is a partition (A, B) of the nodes with $s \in A$ and $t \in B$.

Def. Its *capacity* is the sum of the capacities of the edges from A to B .

$$\text{cap}(A, B) = \sum_{e \text{ out of } A} c(e)$$

Min-cut problem. Find a cut of minimum capacity.

OBIETTIVO

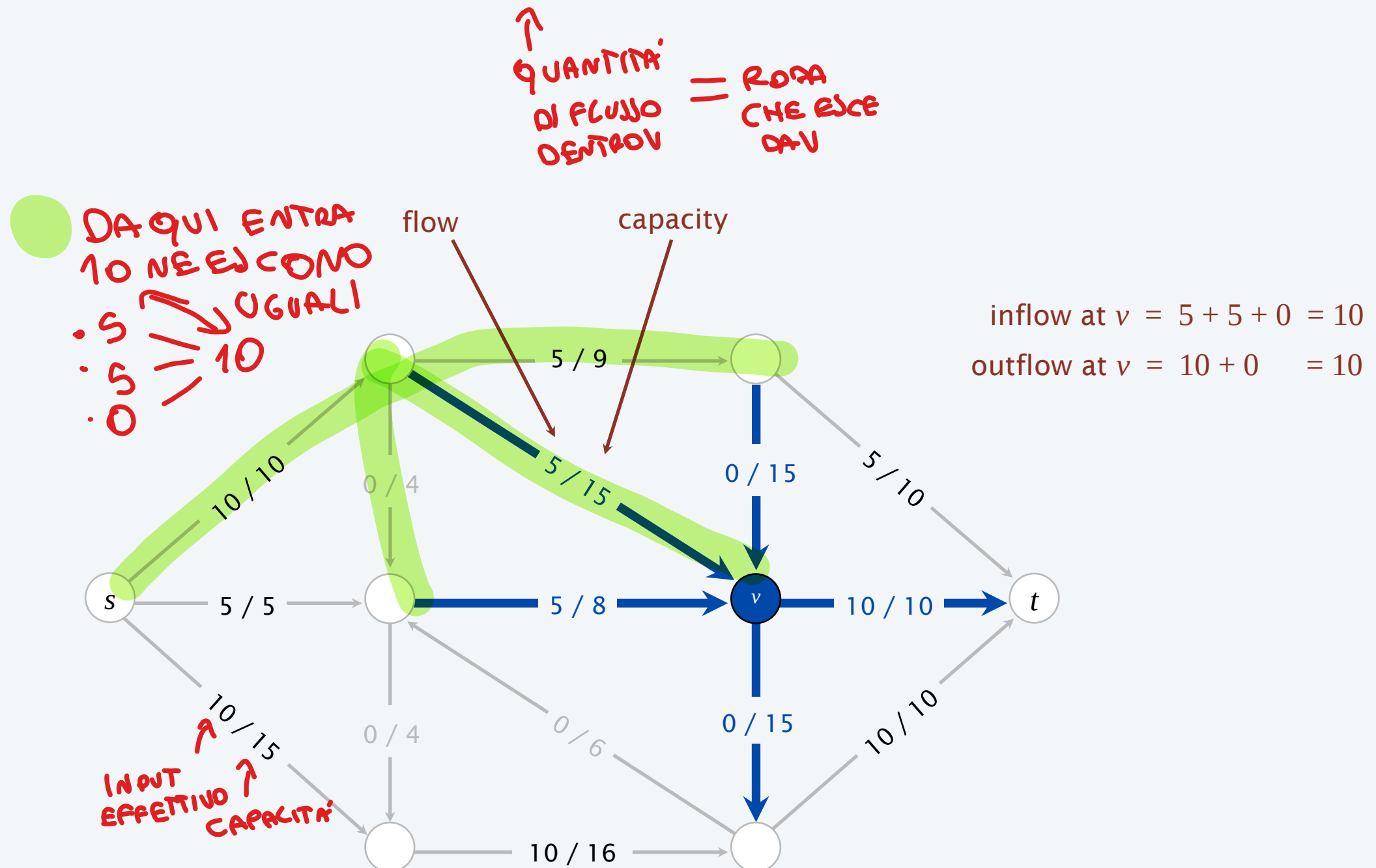


capacity = $10 + 8 + 10 = 28$

Maximum-flow problem

Def. An *st-flow* (flow) f is a function that satisfies:

- For each $e \in E$: $0 \leq f(e) \leq c(e)$ [capacity]
- For each $v \in V - \{s, t\}$: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ [flow conservation]



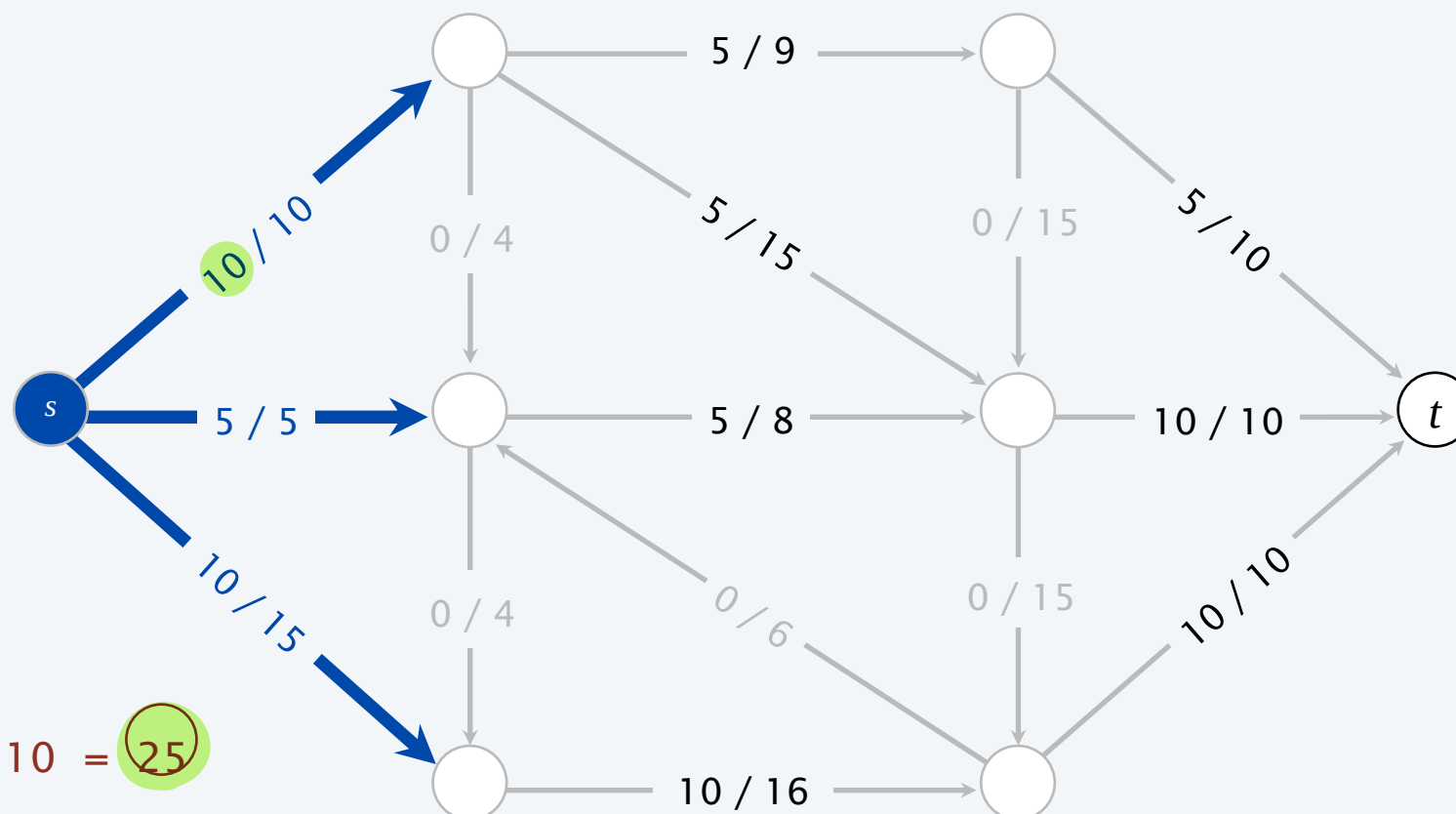
Maximum-flow problem

Def. An *st-flow* (flow) f is a function that satisfies:

- For each $e \in E$: $0 \leq f(e) \leq c(e)$ [capacity]
- For each $v \in V - \{s, t\}$: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ [flow conservation]

Def. The *value* of a flow f is: $val(f) = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ in to } s} f(e)$

$\uparrow$ USCENTI $\uparrow$ ENTRANTI
FLUSSO NETTODAS



$$\text{value} = 5 + 10 + 10 = 25$$

IN QUESTO ESEMPIO

USCENTI = 25
ENTRANTI = 0

$$25 - 0 = 25$$

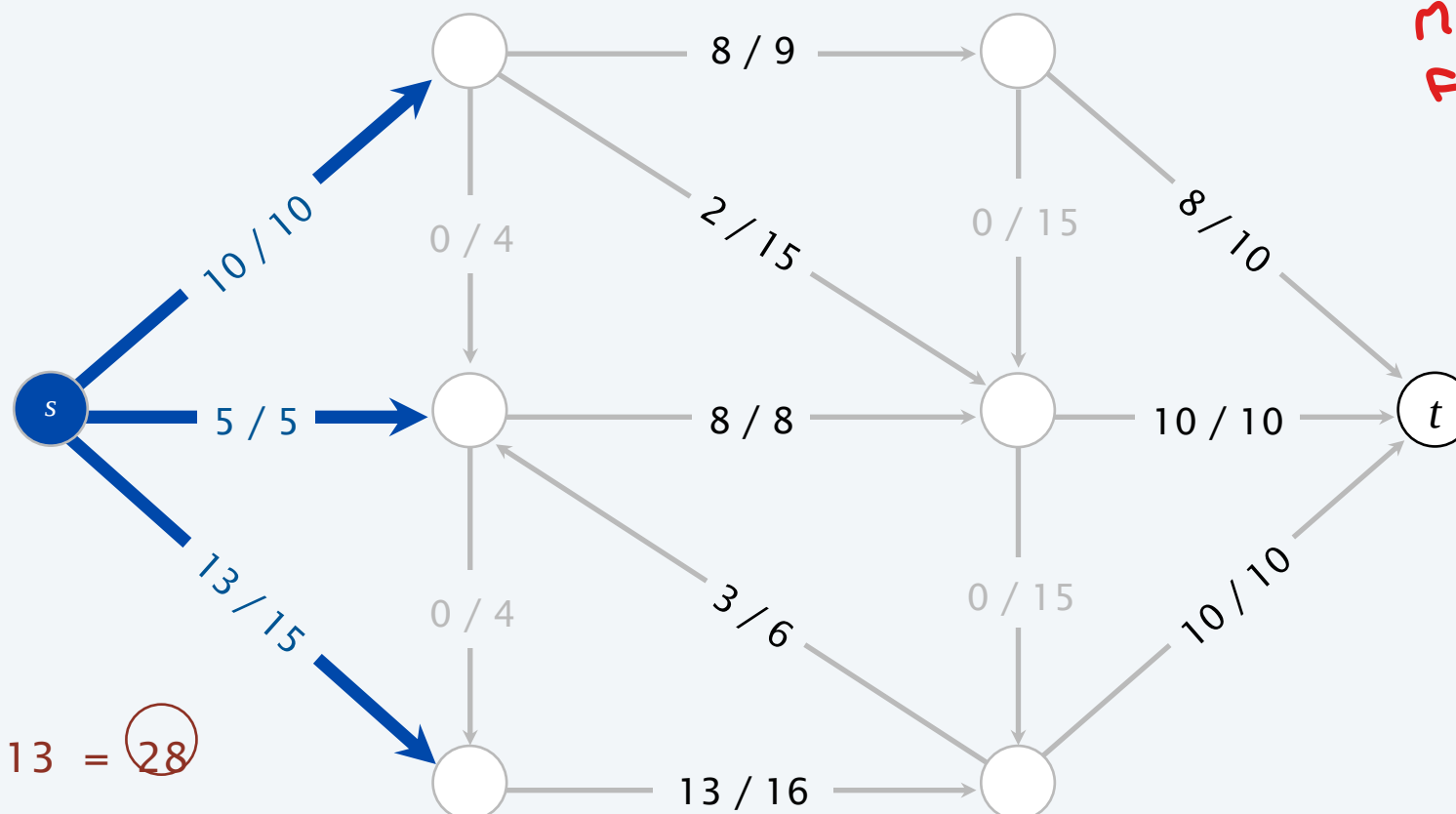
Maximum-flow problem

Def. An *st-flow* (flow) f is a function that satisfies:

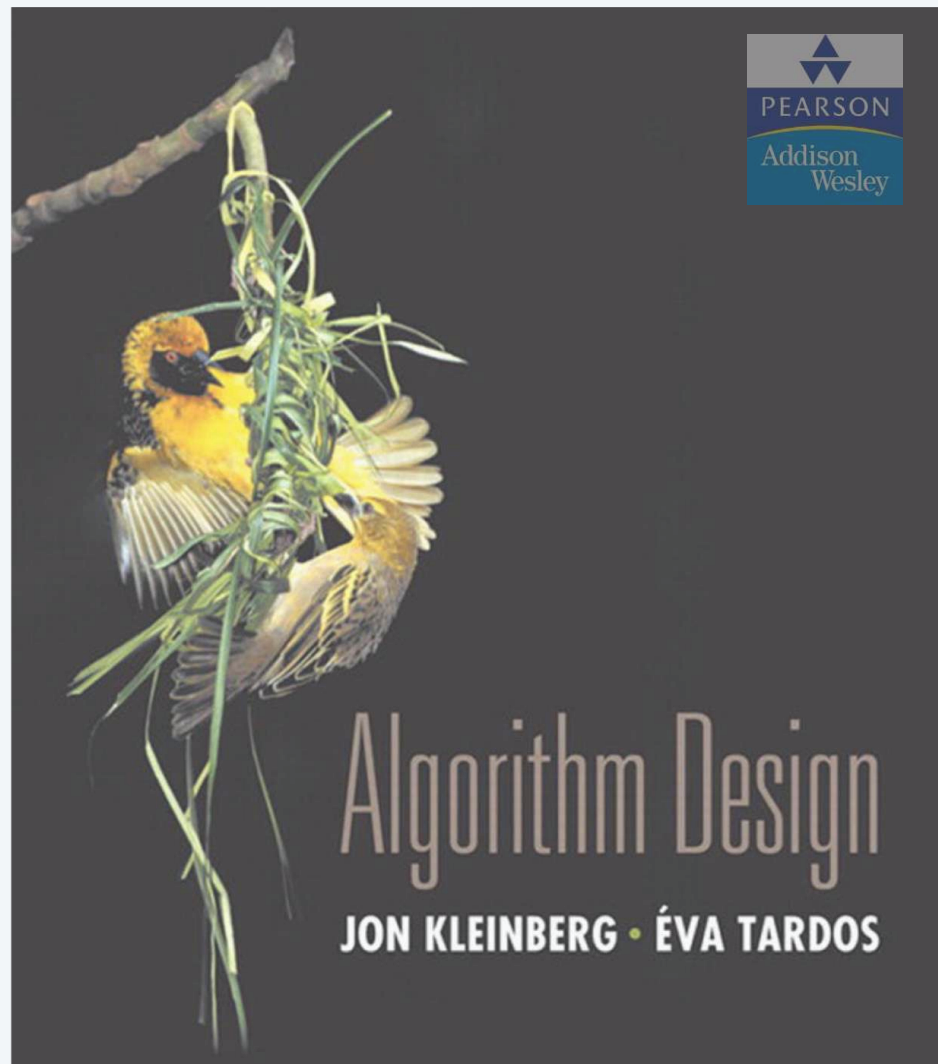
- For each $e \in E$: $0 \leq f(e) \leq c(e)$ [capacity]
- For each $v \in V - \{s, t\}$: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ [flow conservation]

Def. The *value* of a flow f is: $val(f) = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ in to } s} f(e)$

Max-flow problem. Find a flow of maximum value. ↪ VOGLIAMO
SPOSTARE PIU' FLUSSO
POSSIBILE
MODIFICHIAMO IL
FLOW PER FARLO



$$\text{value} = 10 + 5 + 13 = 28$$



SECTION 7.1

7. NETWORK FLOW I

- *max-flow and min-cut problems*
- **Ford–Fulkerson algorithm**
- *max-flow min-cut theorem*
- *choosing good augmenting paths*

Toward a max-flow algorithm

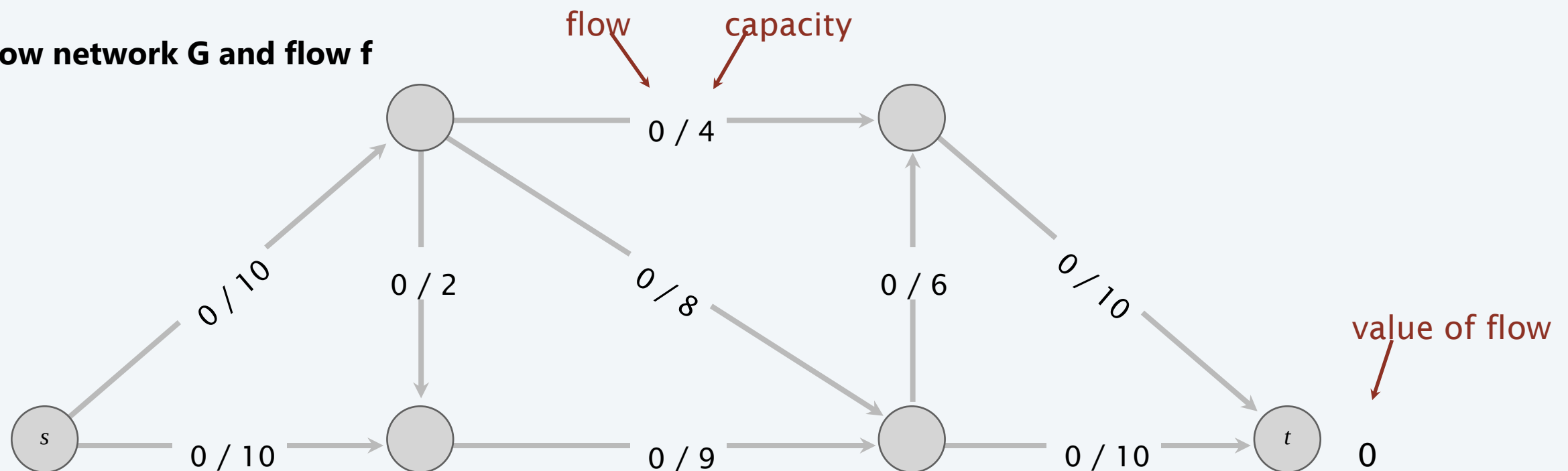
Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

NON HO ARCHI CON $\text{FLOW} > \text{CAPAC}$.
 $\Rightarrow$ SICURO
E' AMMISSIBILE

TUTTI I FLOW SONO A 0
QUANTO METTO?

flow network G and flow f



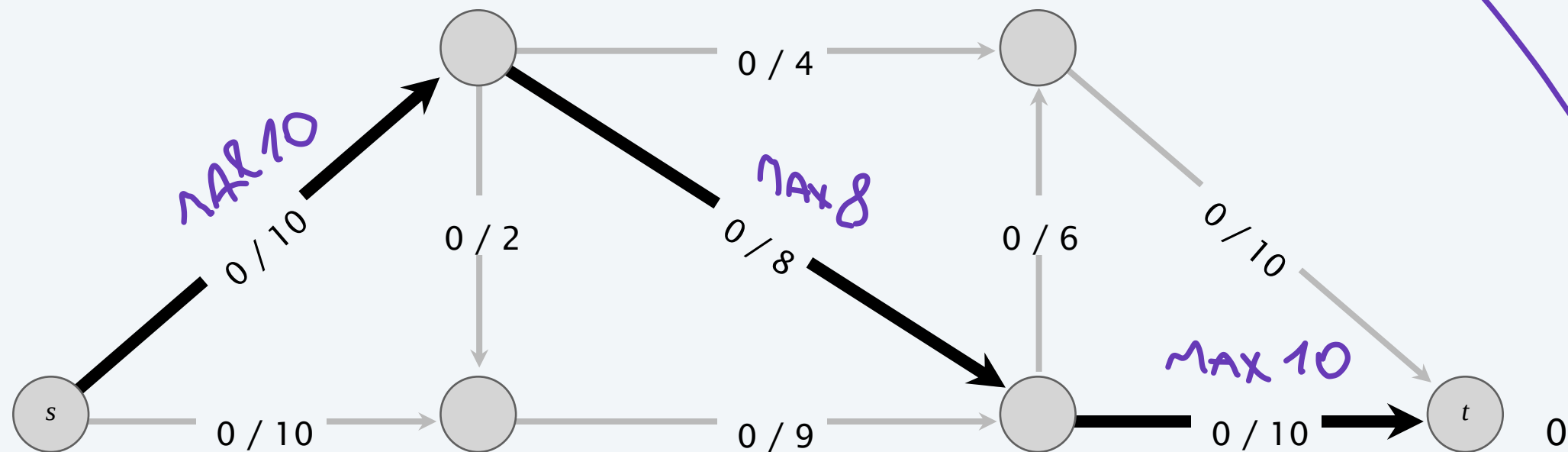
Toward a max-flow algorithm

Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

PRENDOUN CAMMINO
DA $s \rightsquigarrow t$, QUANTO FLUSSO?
• ORA O

flow network G and flow f



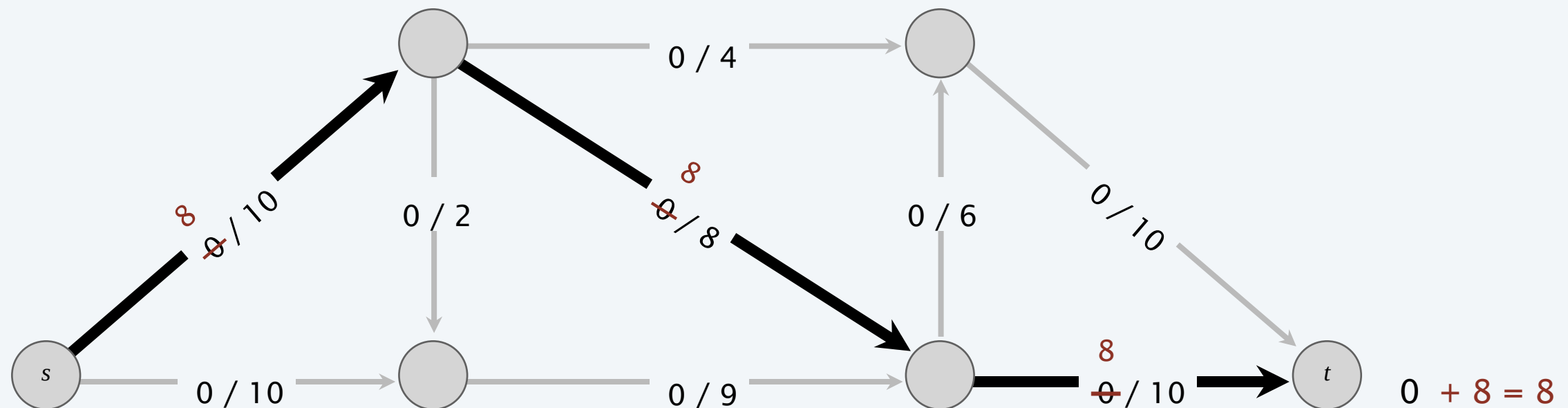
Toward a max-flow algorithm

Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- **Augment flow along path P .**
- Repeat until you get stuck.

AUMENTO IL FLOW
(ON CAPACITÀ
DI BOTTLENECK
(PRENDI MINIMO))

flow network G and flow f

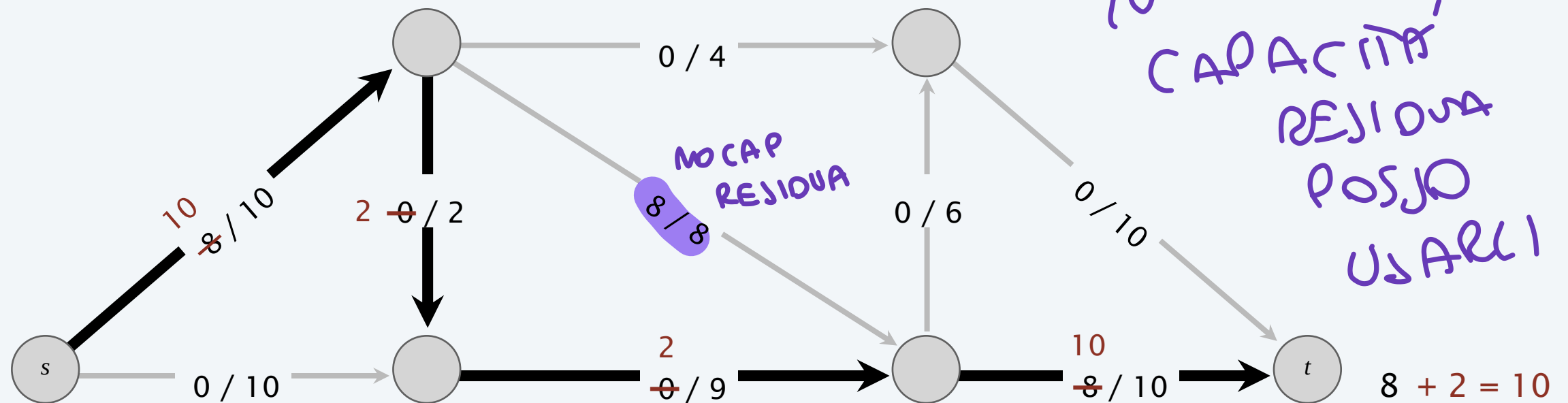


Toward a max-flow algorithm

Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

flow network G and flow f

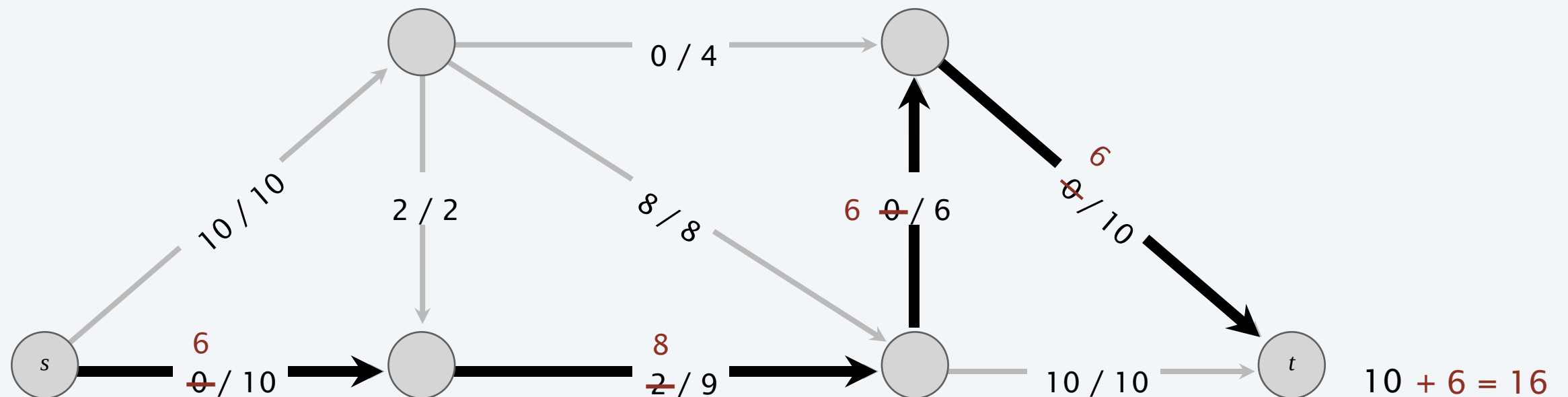


Toward a max-flow algorithm

Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

flow network G and flow f

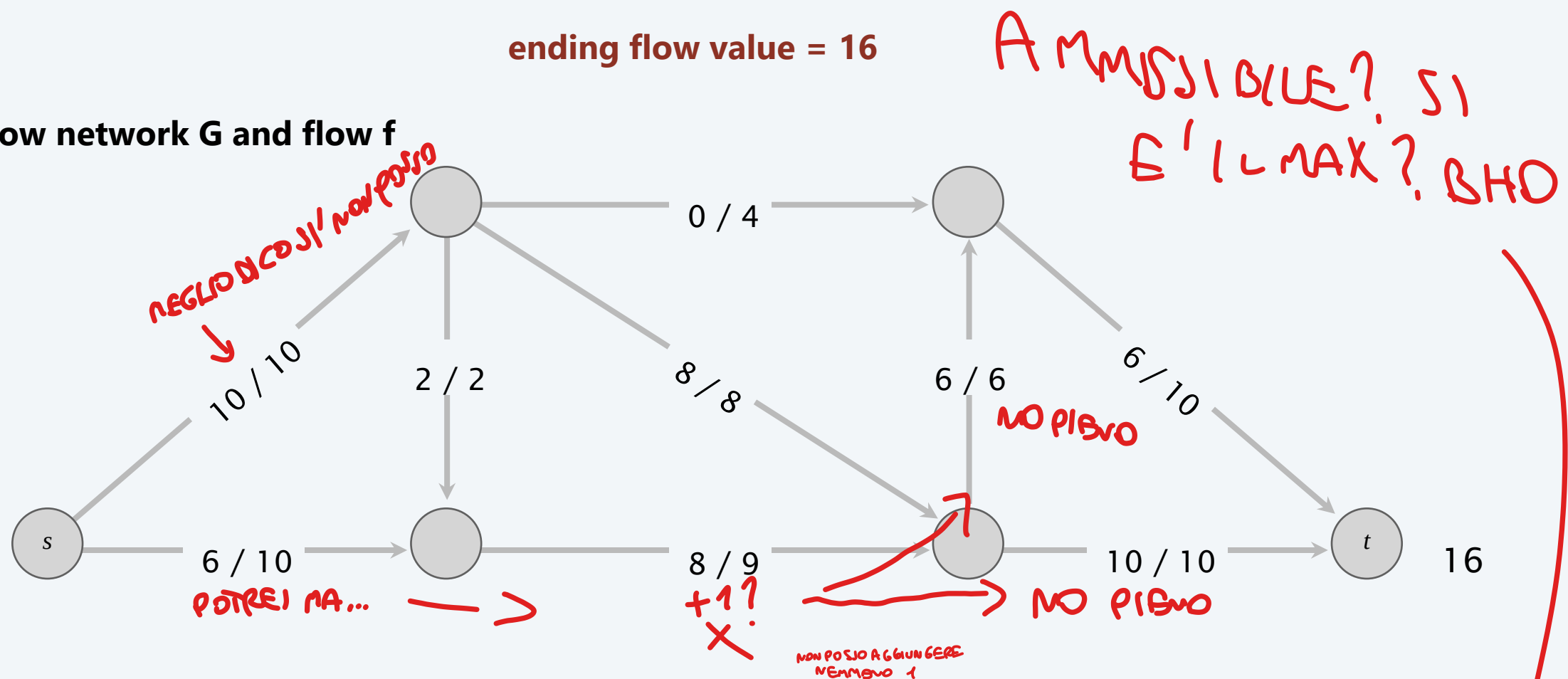


Toward a max-flow algorithm

Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

flow network G and flow f



Toward a max-flow algorithm

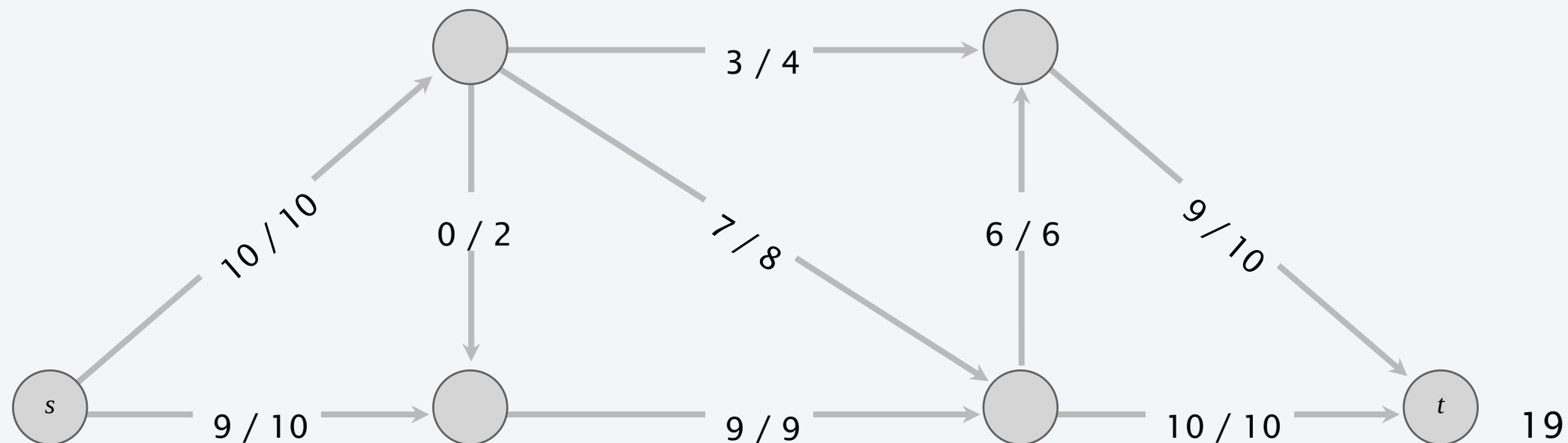
Greedy algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
- Augment flow along path P .
- Repeat until you get stuck.

NON ERALMAX
QUESTO ALGORITMO GREEDY
NON RITROVA IL MAX

but max-flow value = 19

flow network G and flow f



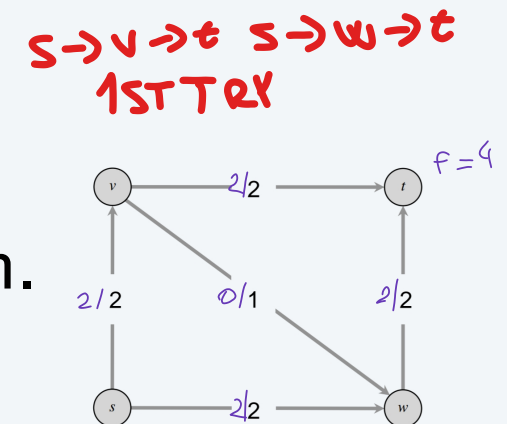
Why the greedy algorithm fails

Q. Why does the greedy algorithm fail?

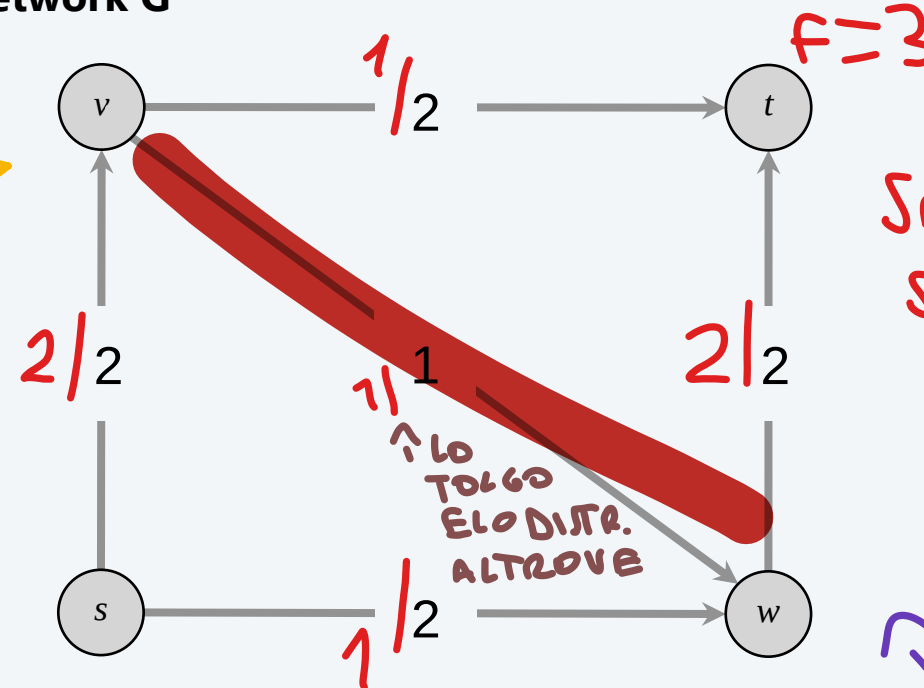
A. Once greedy algorithm increases flow on an edge, it never decreases it.

Ex. Consider flow network G .

- The unique max flow f^* has $f^*(v, w) = 0$.
- Greedy algorithm could choose $s \rightarrow v \rightarrow w \rightarrow t$ as first path.



flow network G



ci tengo a ricordare che gli archi entranti da un nodo devono essere uguali a quelli uscenti

SEPARATO DA QUA
SO CAZZI

VOGLIO UN MECCANISMO
PER FARE ALTRI TENTATIVI
PER VEDERE SE $val(t)$
RIGUARD

POSSIAMO
VEDERE CHE IN BASE A CUI L'ALG

SCEGLIE POSSIAMO
IN L'ORDINE
IN CUI
NON-BEST

Bottom line. Need some mechanism to “undo” a bad decision.

Residual network

Original edge. $e = (u, v) \in E$.

- Flow $f(e)$.
- Capacity $c(e)$.

Reverse edge. $e^{\text{reverse}} = (v, u)$.

- “Undo” flow sent.

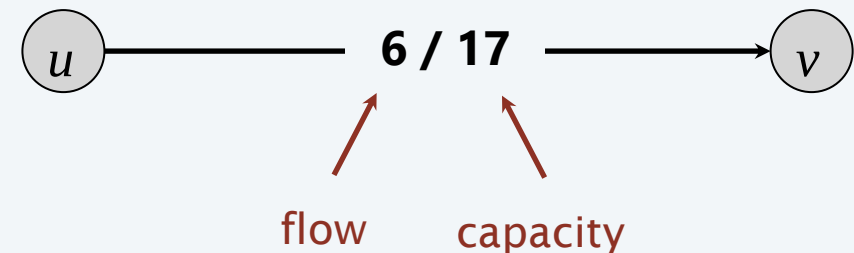
Residual capacity.

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E \\ f(e^{\text{reverse}}) & \text{if } e^{\text{reverse}} \in E \end{cases}$$

Residual network. $G_f = (V, E_f, s, t, c_f)$.

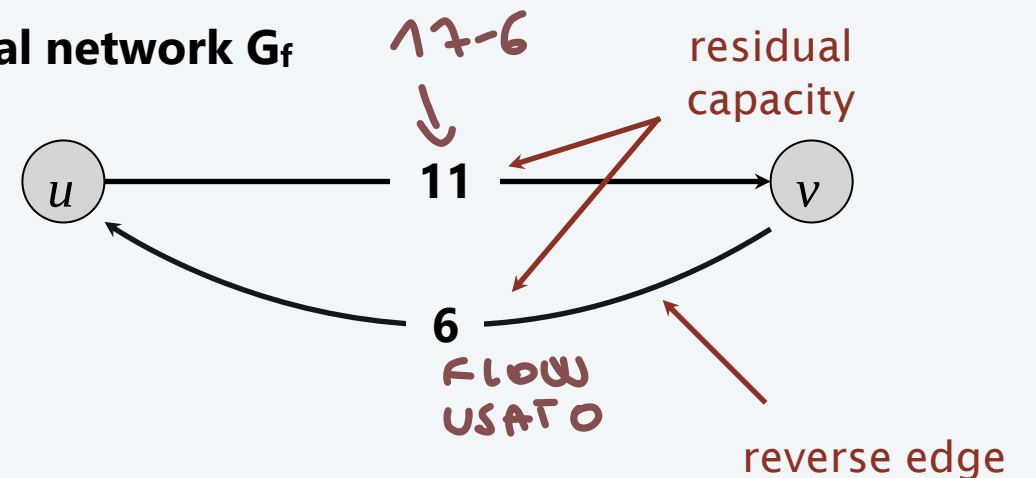
- $E_f = \{e : f(e) < c(e)\} \cup \{e : f(e^{\text{reverse}}) > 0\}$.
- Key property: f' is a flow in G_f iff $f + f'$ is a flow in G .

original flow network G



"non basta semplicemente fare una sottrazione?"
La risposta breve è:
Sì, in teoria potresti "sottrarre" a mano, ma il reverse edge serve per rendere questa sottrazione parte integrante dell'algoritmo e generalizzarla automaticamente.

residual network G_f

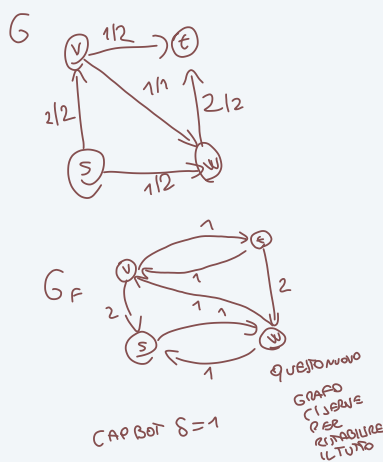
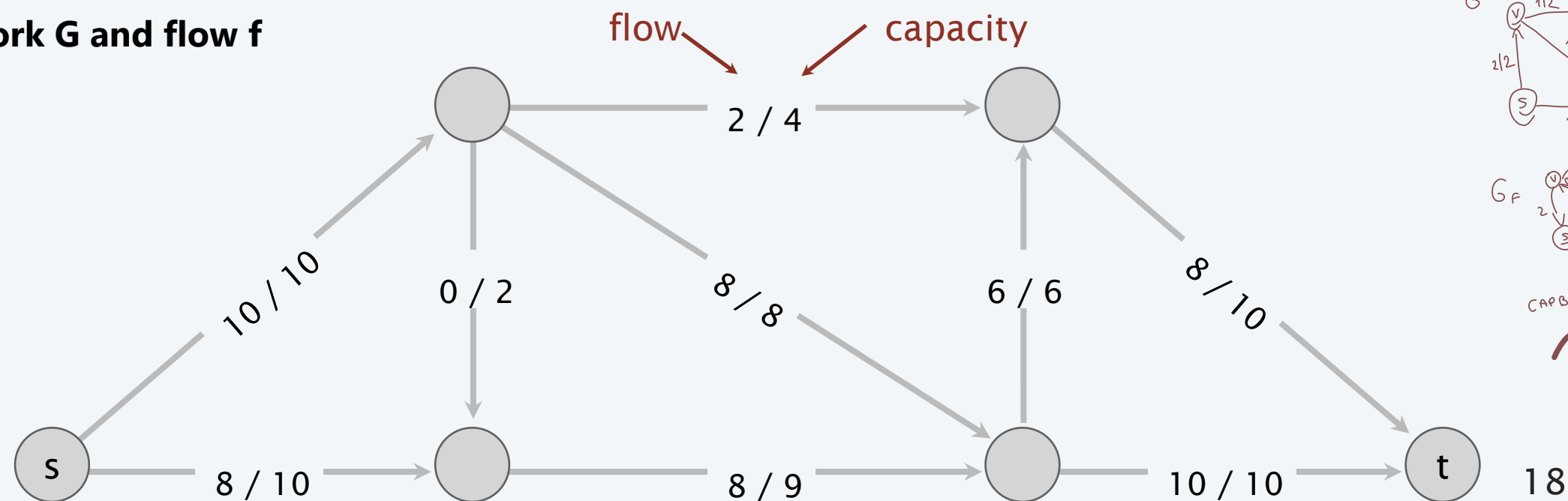


edges with positive residual capacity

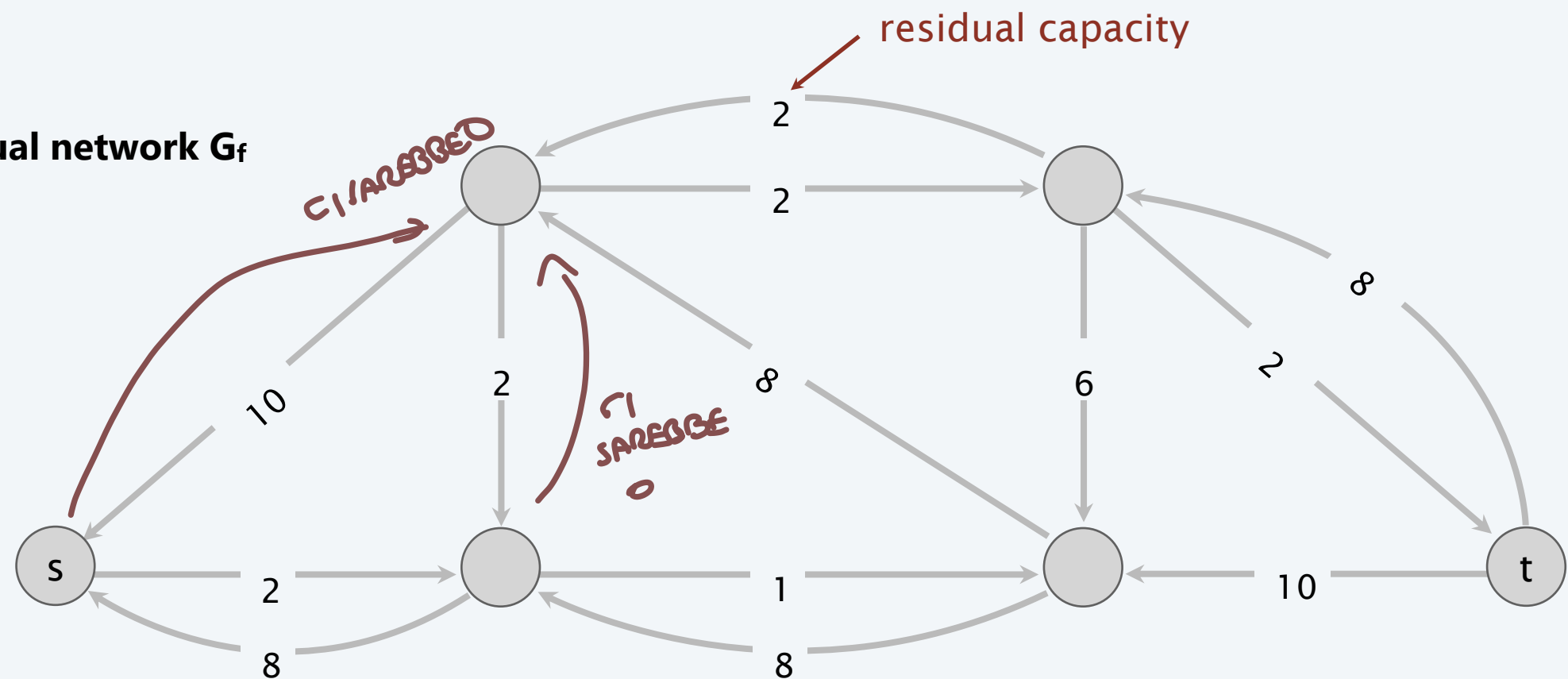
where flow on a reverse edge negates flow on corresponding forward edge

Residual network: an example

network G and flow f



residual network G_f



Augmenting path

Def. An **augmenting path** is a simple $s \rightsquigarrow t$ path in the residual network G_f .

CAMMINO DA S A T S U R E T E R E S I D U A

Def. The **bottleneck capacity** of an augmenting path P is the minimum residual capacity of any edge in P .

CAP. RESIDUA + PICCOLA

Key property. Let f be a flow and let P be an augmenting path in G_f .

Then, after calling $f' \leftarrow \text{AUGMENT}(f, c, P)$, the resulting f' is a flow and

$\text{val}(f') = \text{val}(f) + \text{bottleneck}(G_f, P)$.

PROVA D'AGGIUNGERE

BOTTLENECK =
MIN(CAP. ARCHI
PATH P)

AUGMENT(f, c, P)

$\delta \leftarrow$ bottleneck capacity of augmenting path P .

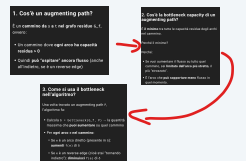
FOREACH edge $e \in P$:

IF ($e \in E$) $f(e) \leftarrow f(e) + \delta$.

ELSE $f(e^{\text{reverse}}) \leftarrow f(e^{\text{reverse}}) - \delta$.

RETURN f .

IL BOTTL.
AGLI ARCHI
ORIGINALI
E LO TOLGO
DA QUELLI
FAKE



Ford–Fulkerson algorithm

Ford–Fulkerson augmenting path algorithm.

- Start with $f(e) = 0$ for each edge $e \in E$.
- Find an $s \rightsquigarrow t$ path P in the residual network G_f .
- Augment flow along path P .
- Repeat until you get stuck.

FORD–FULKERSON(G)

FOREACH edge $e \in E$: $f(e) \leftarrow 0$.

$G_f \leftarrow$ residual network of G with respect to flow f .

WHILE (there exists an $s \rightsquigarrow t$ path P in G_f) 

$f \leftarrow$ AUGMENT(f, c, P).

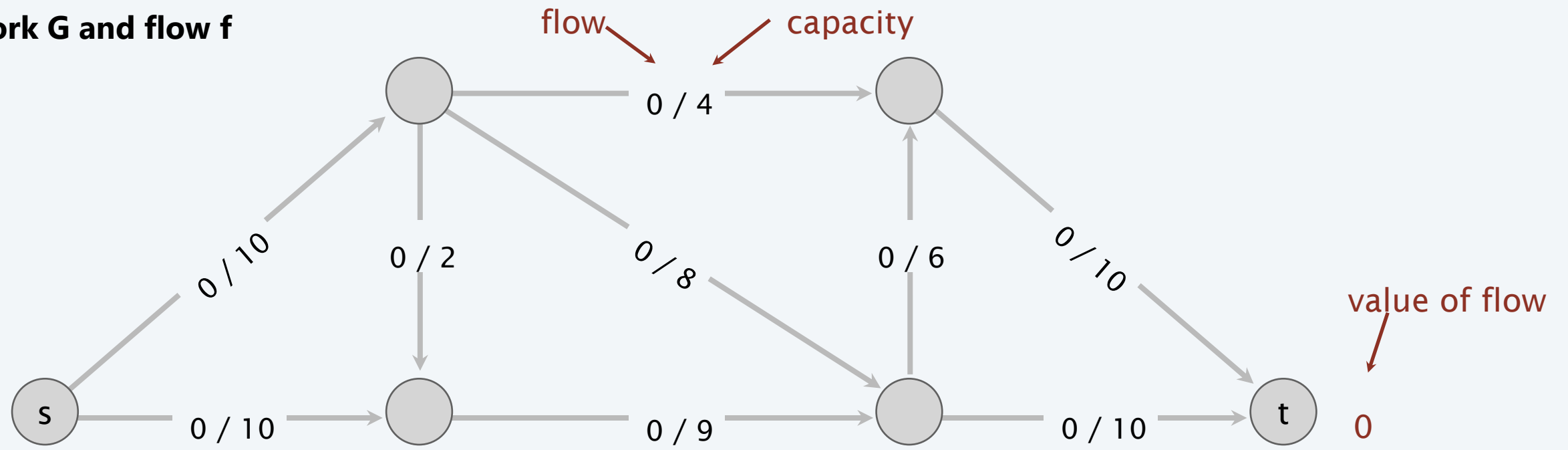
Update G_f .

RETURN f .

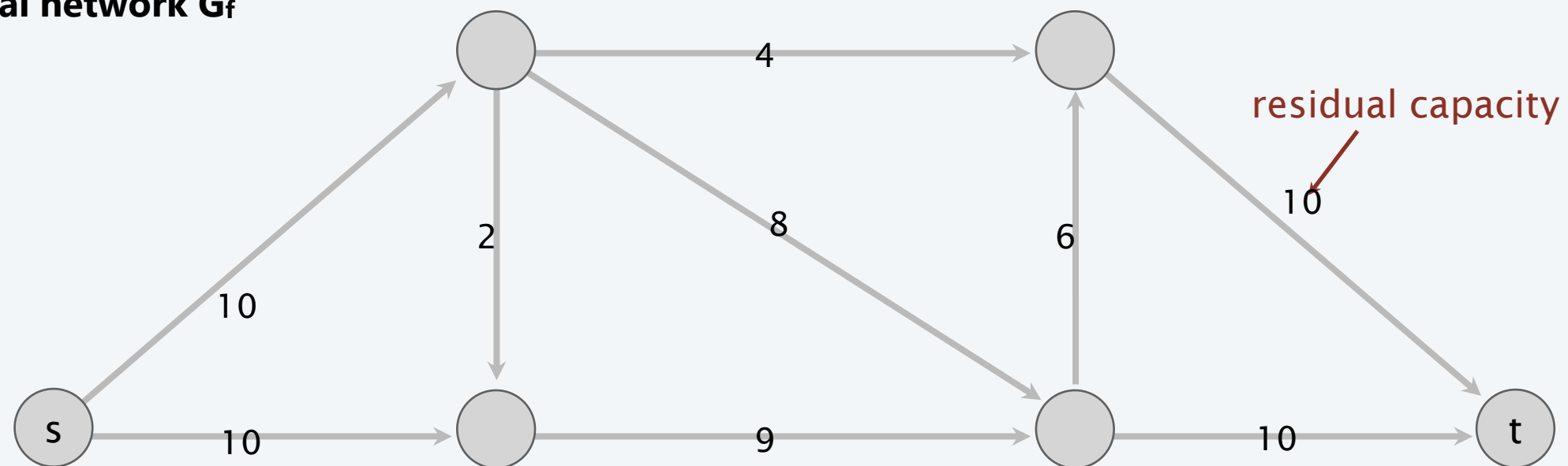
augmenting path

Ford–Fulkerson algorithm demo

network **G** and flow **f**

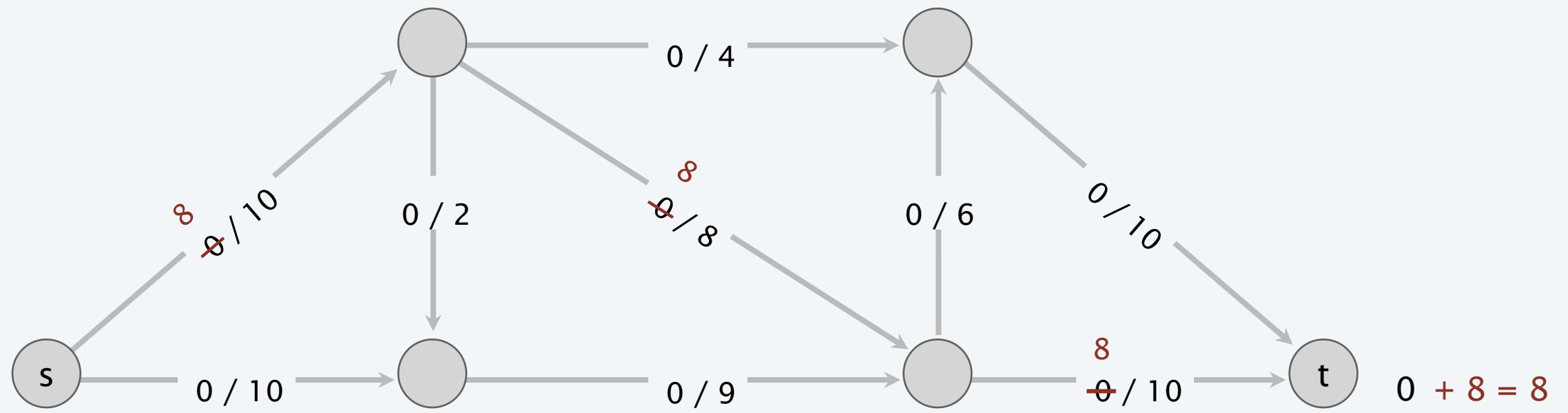


residual network **G_f**

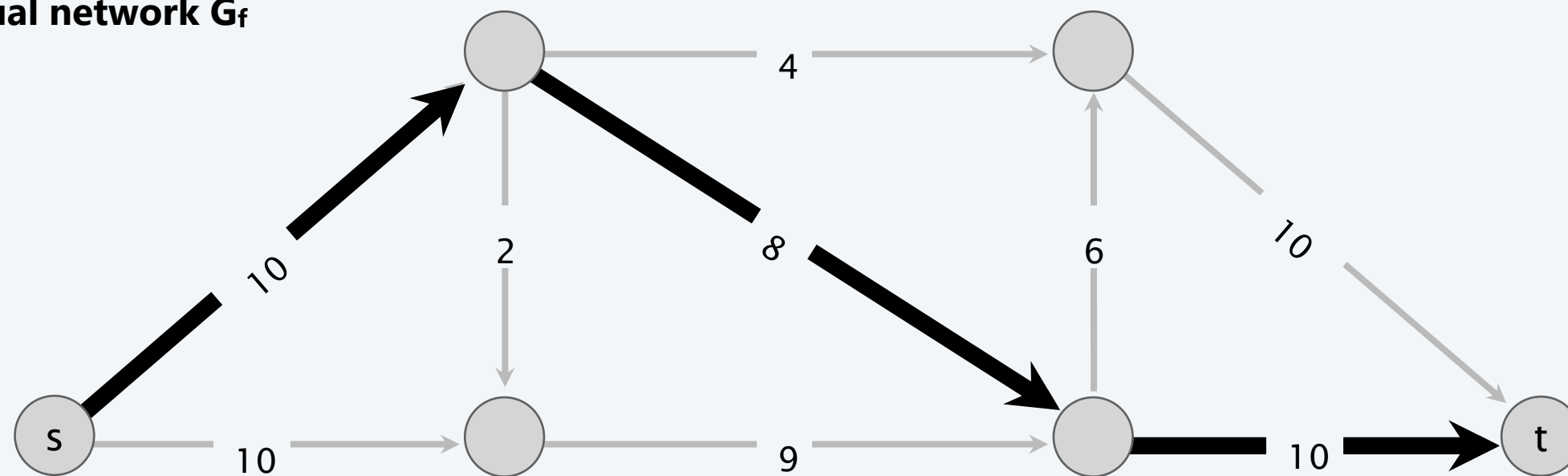


Ford–Fulkerson algorithm demo

network **G** and flow **f**

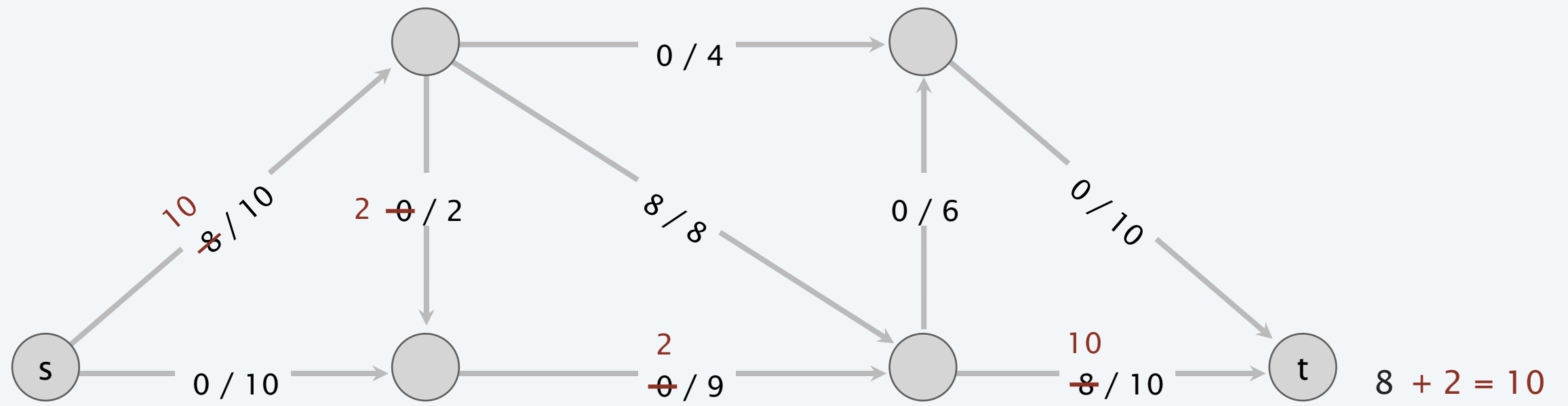


residual network **G_f**

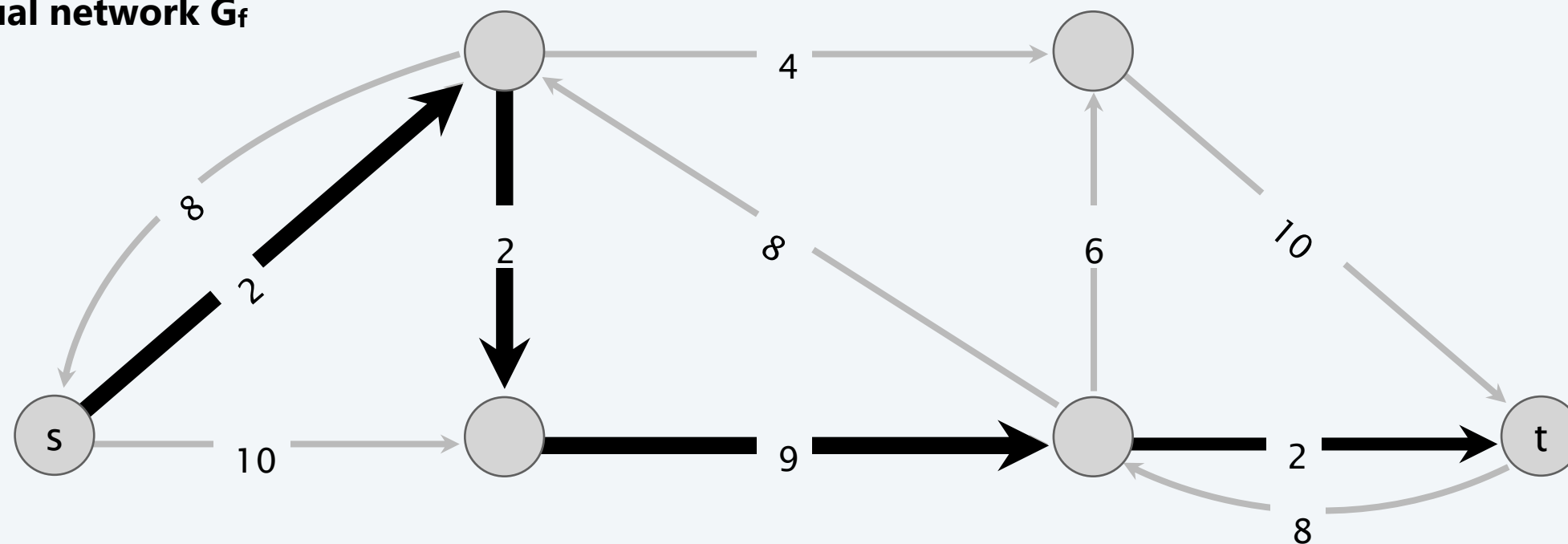


Ford–Fulkerson algorithm demo

network **G** and flow **f**

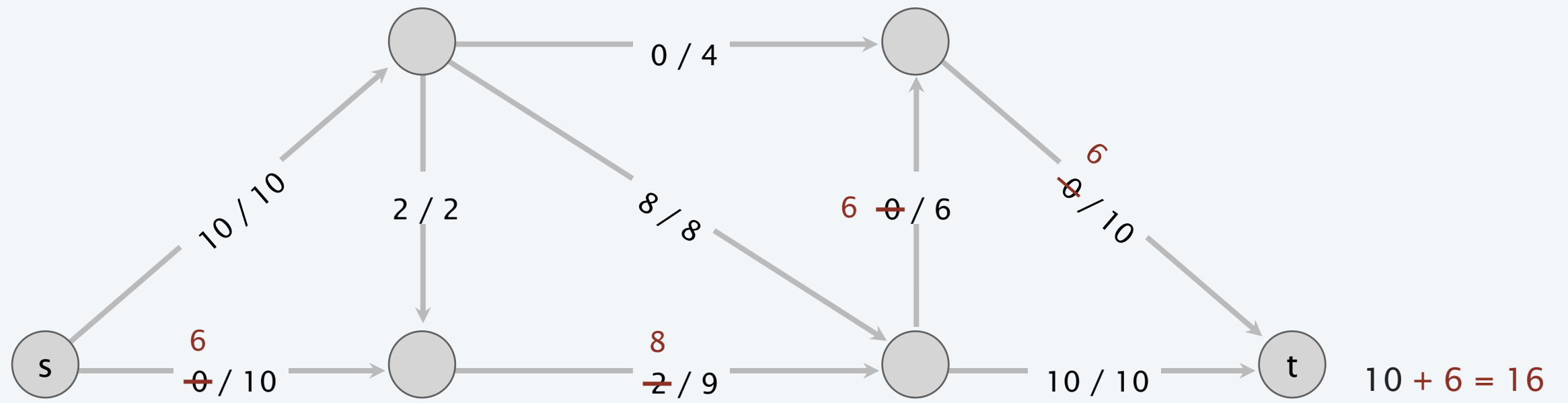


residual network **G_f**

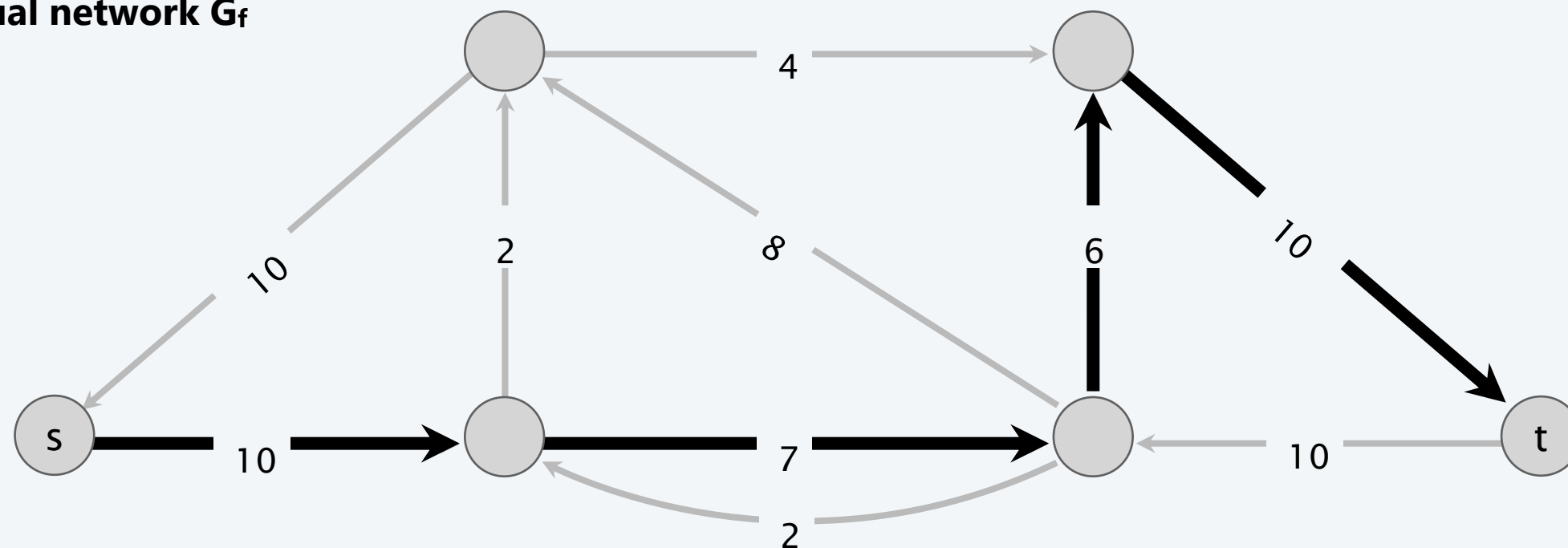


Ford–Fulkerson algorithm demo

network **G** and flow **f**

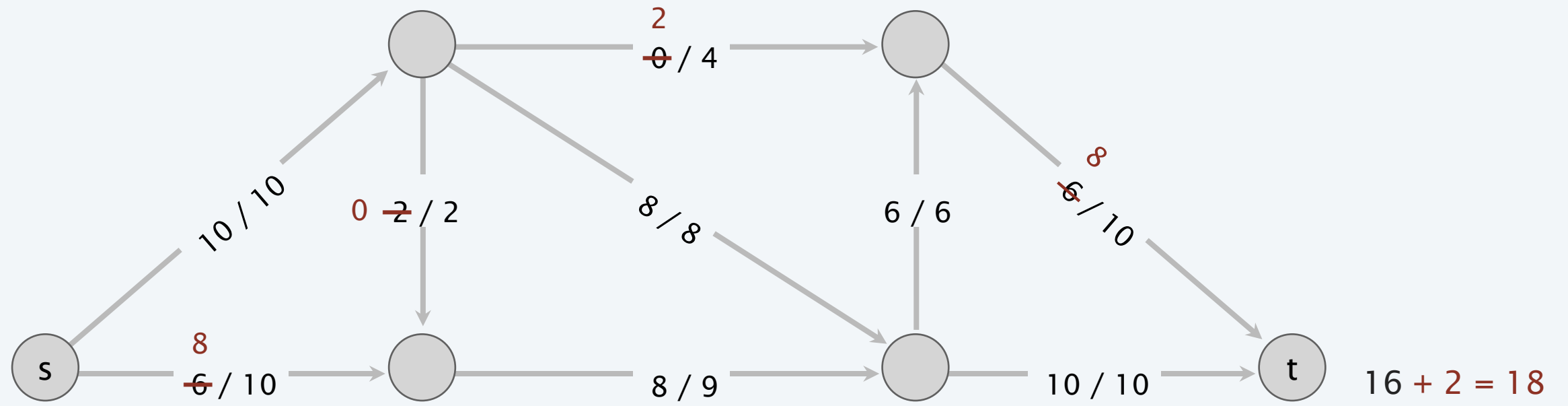


residual network **G_f**

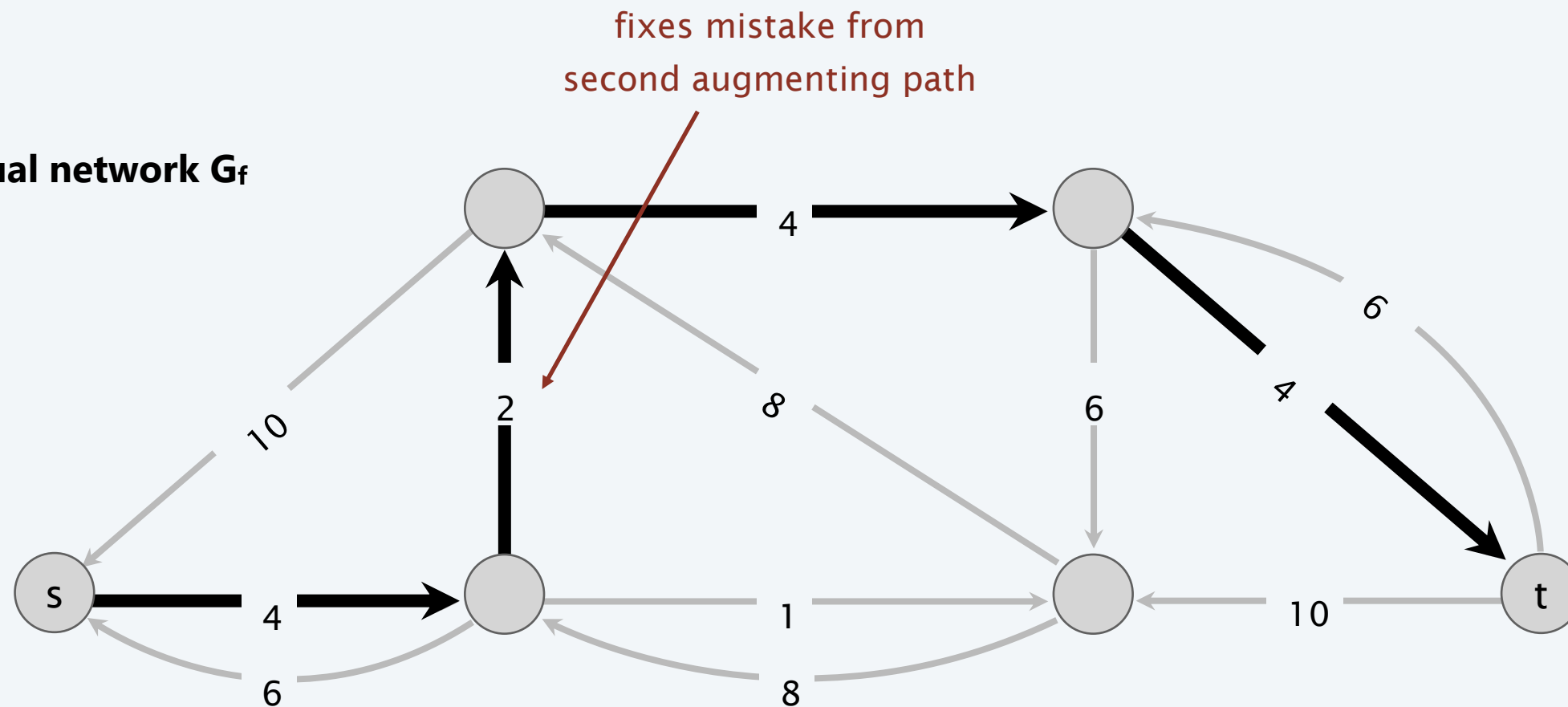


Ford–Fulkerson algorithm demo

network **G** and flow **f**

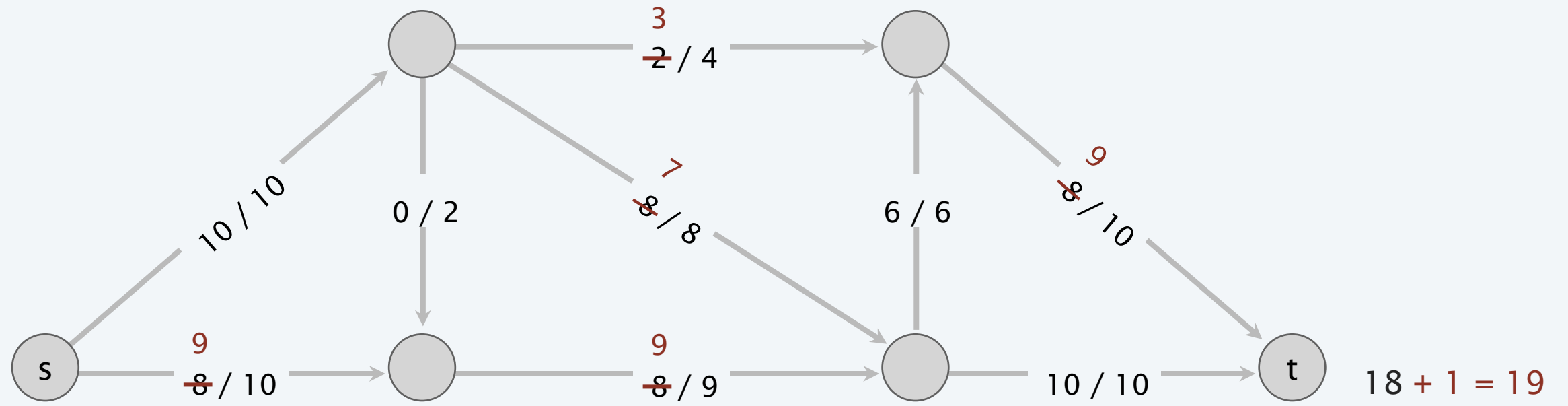


residual network **G_f**

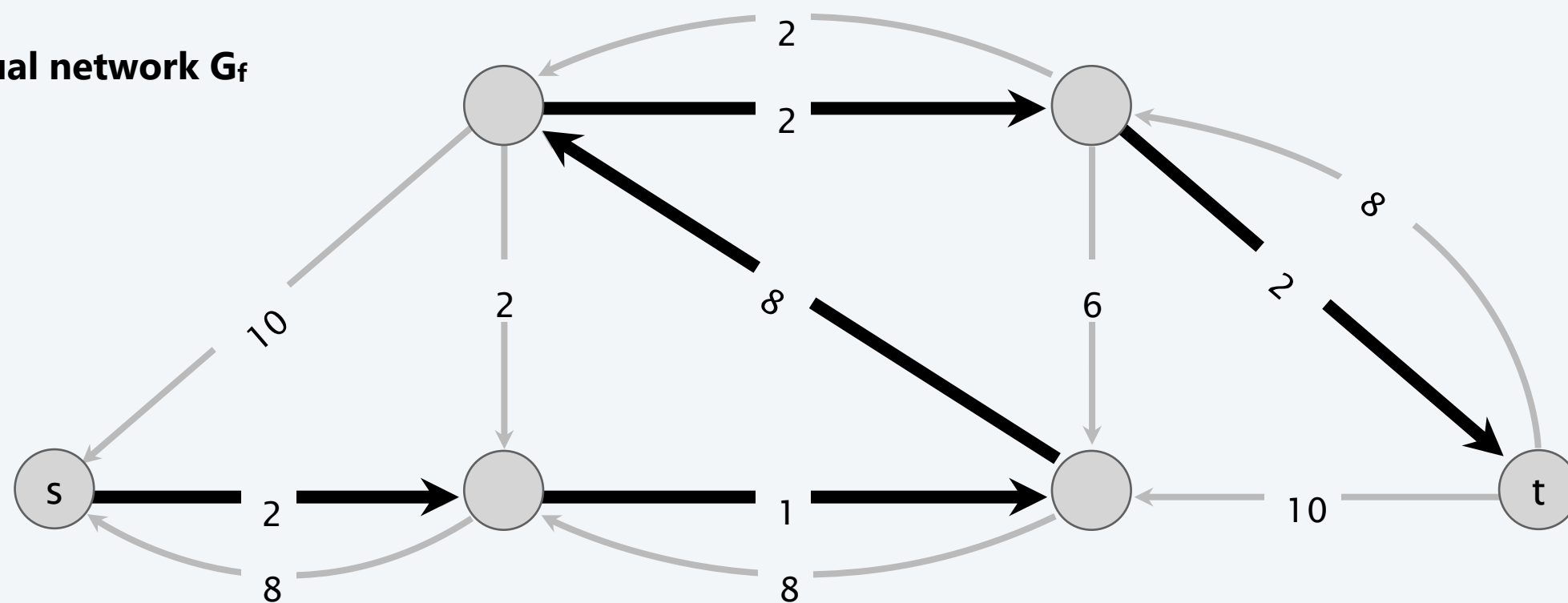


Ford–Fulkerson algorithm demo

network **G** and flow **f**

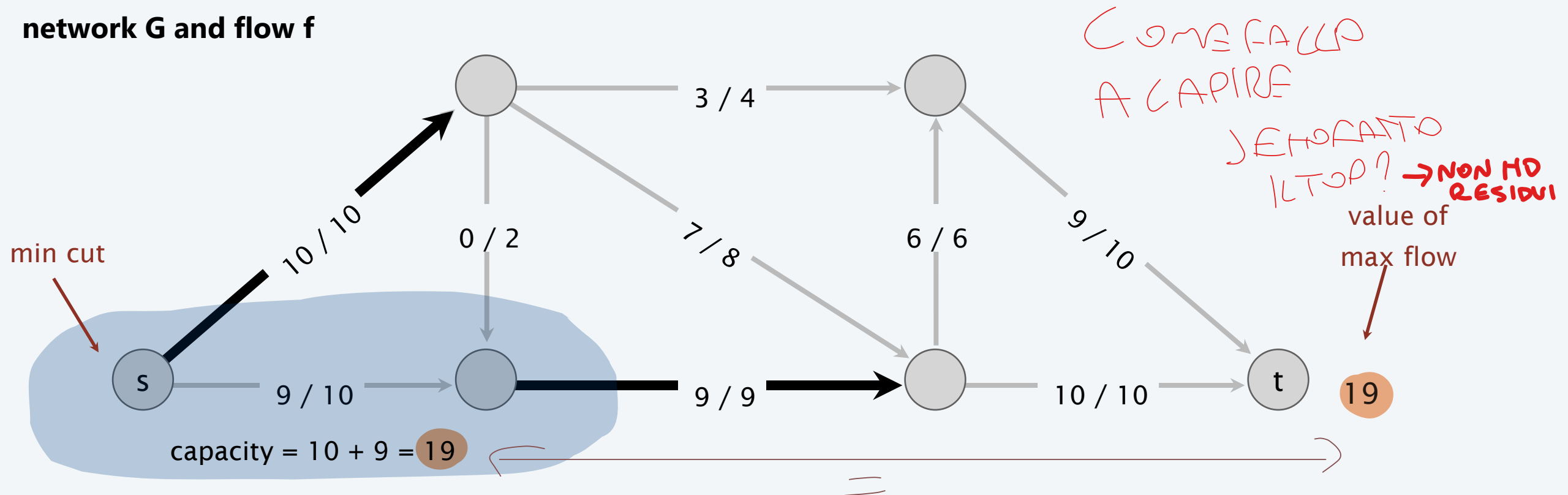


residual network **G_f**

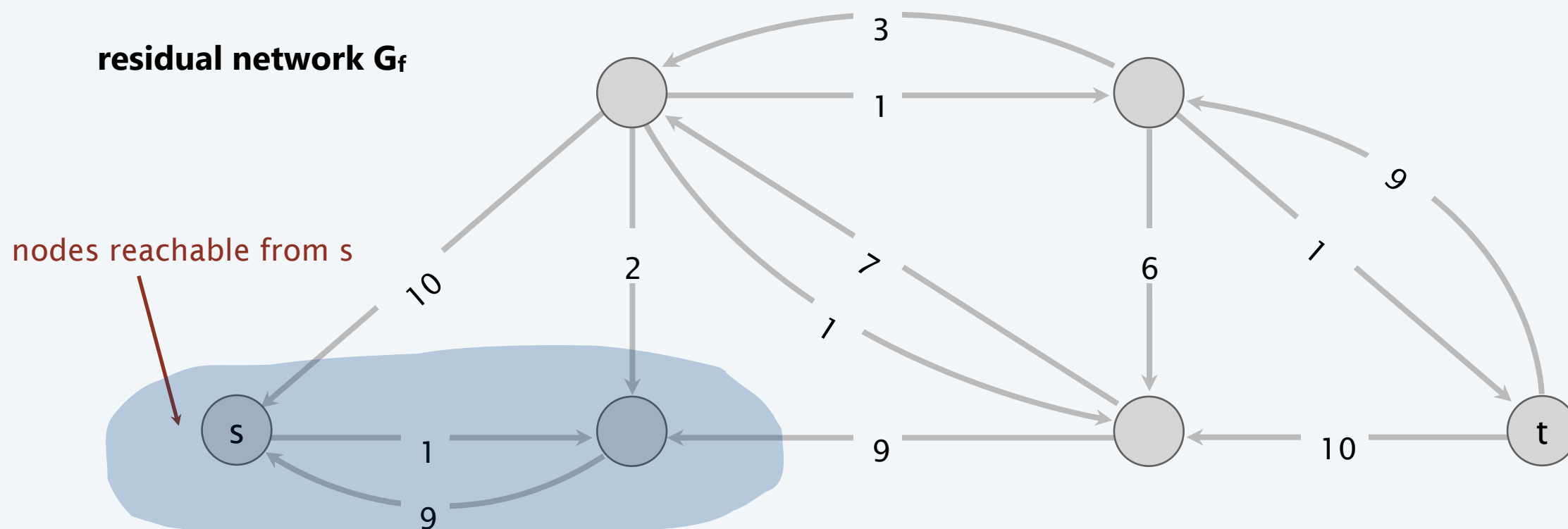


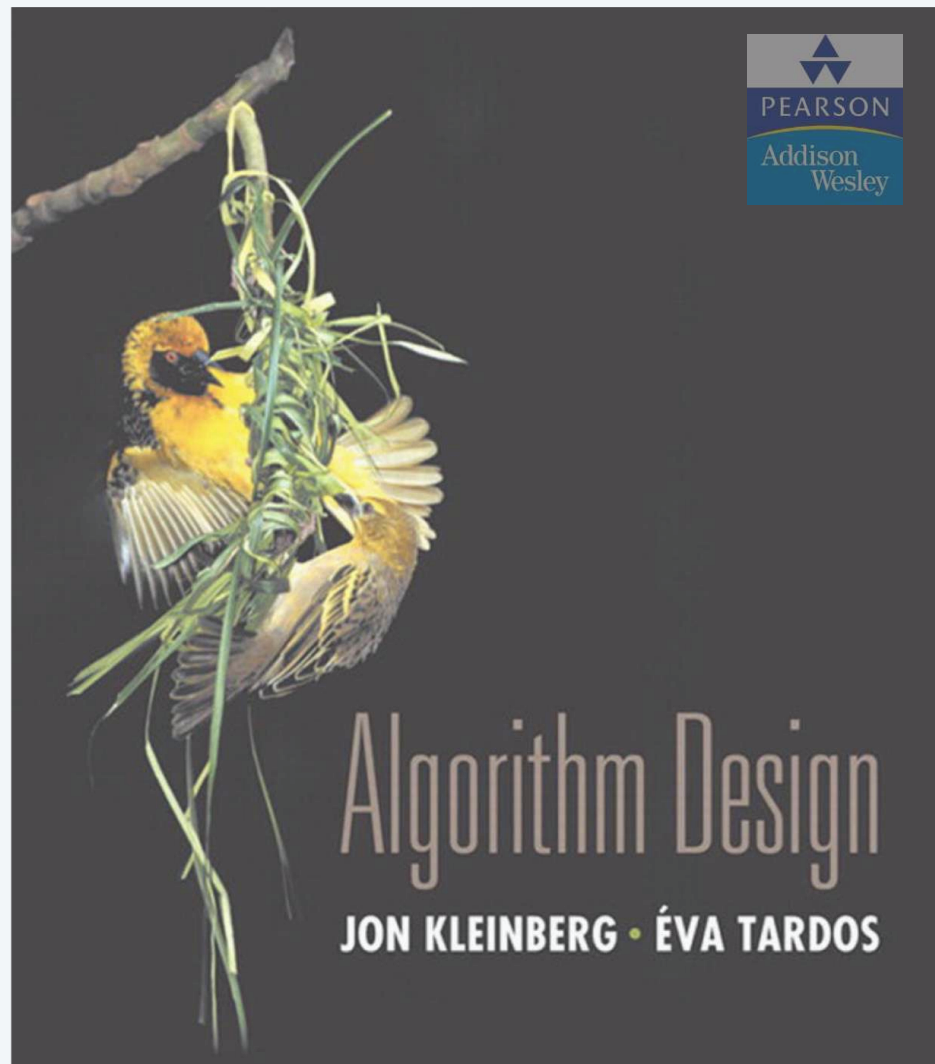
Ford-Fulkerson algorithm demo

network **G** and flow **f**



residual network **G_f**





SECTION 7.2

7. NETWORK FLOW I

- *max-flow and min-cut problems*
- *Ford–Fulkerson algorithm*
- ***max-flow min-cut theorem***
- *choosing good augmenting paths*

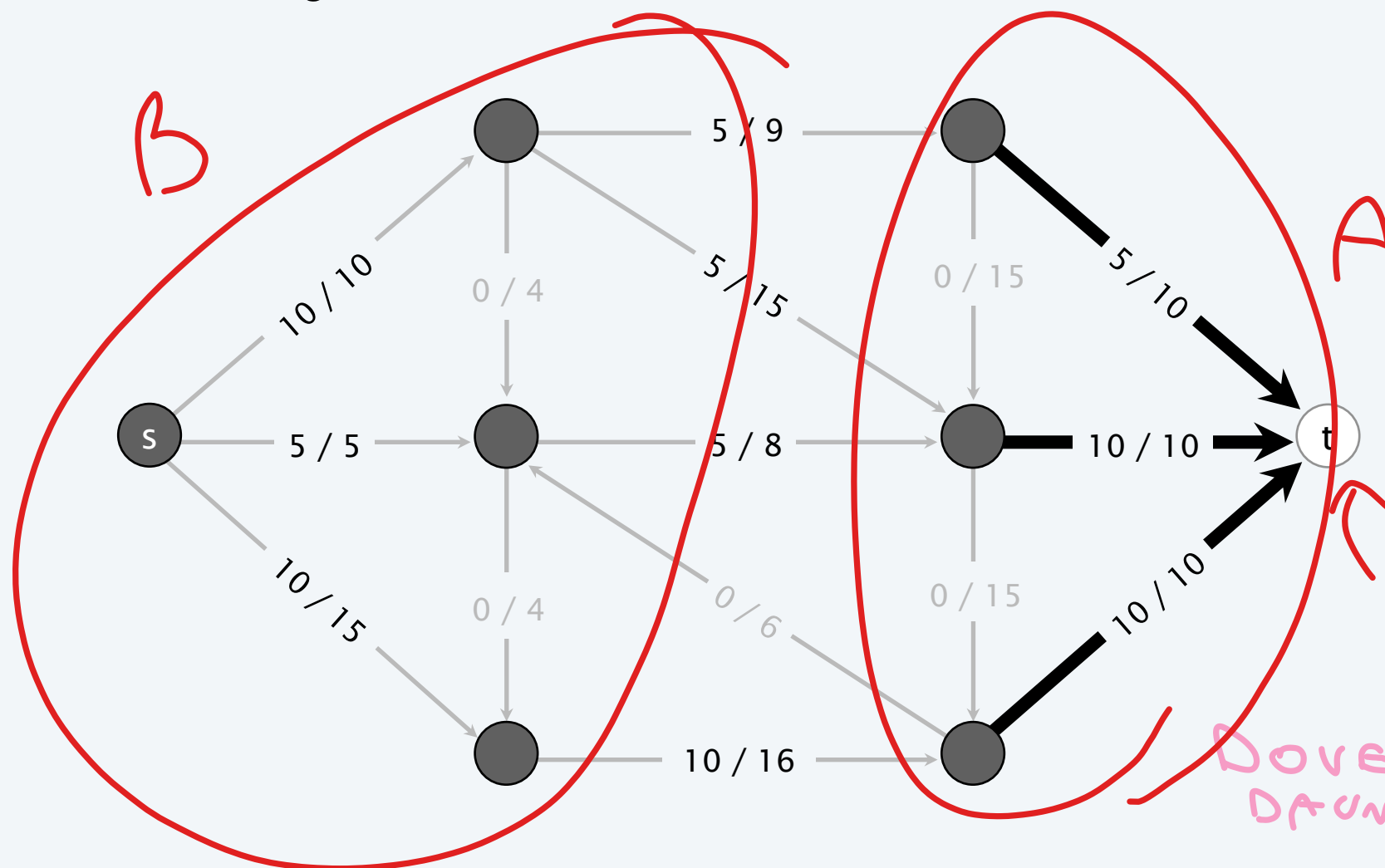
Relationship between flows and cuts

Flow value lemma. Let f be any flow and let (A, B) be any cut. Then, the value of the flow f equals the net flow across the cut (A, B) .

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$

SIA f UN FLUSSO AMMISSIBILE

net flow across cut = 5 + 10 + 10 = 25



Questo "taglio su t " è molto comodo per verificare rapidamente il valore del flusso, ma non è necessariamente il taglio minimo. È solo un esempio di s - t cut, utile per applicare il flow value lemma.

value of flow = 25

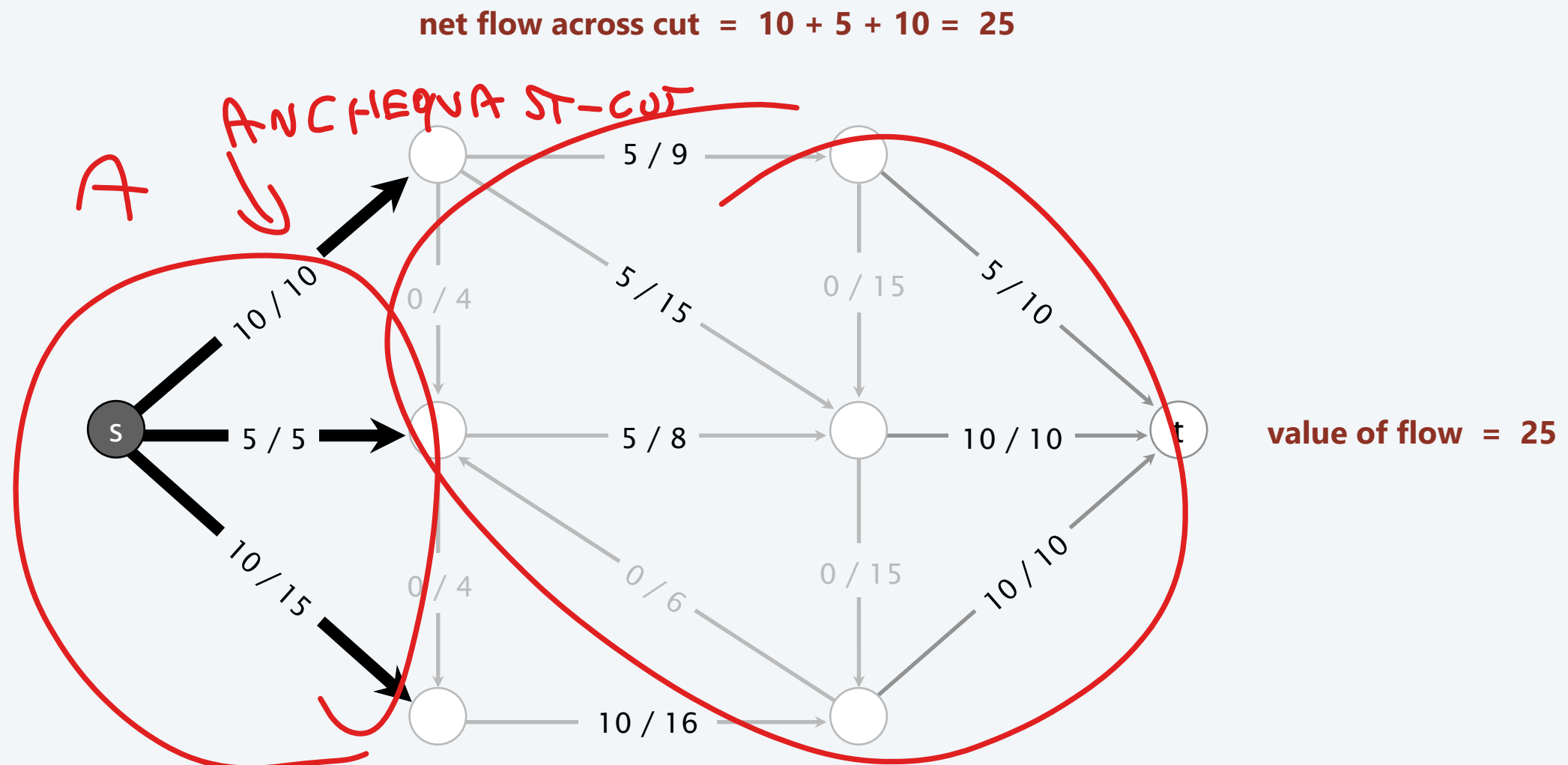
QUI
E' COME
SE SI
FACEVA UN
ST-CUT

DOVE HO E
DA UNA PARTE E TUTTO IL RESTO
DA UN'ALTRA

Relationship between flows and cuts

Flow value lemma. Let f be any flow and let (A, B) be any cut. Then, the value of the flow f equals the net flow across the cut (A, B) .

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$



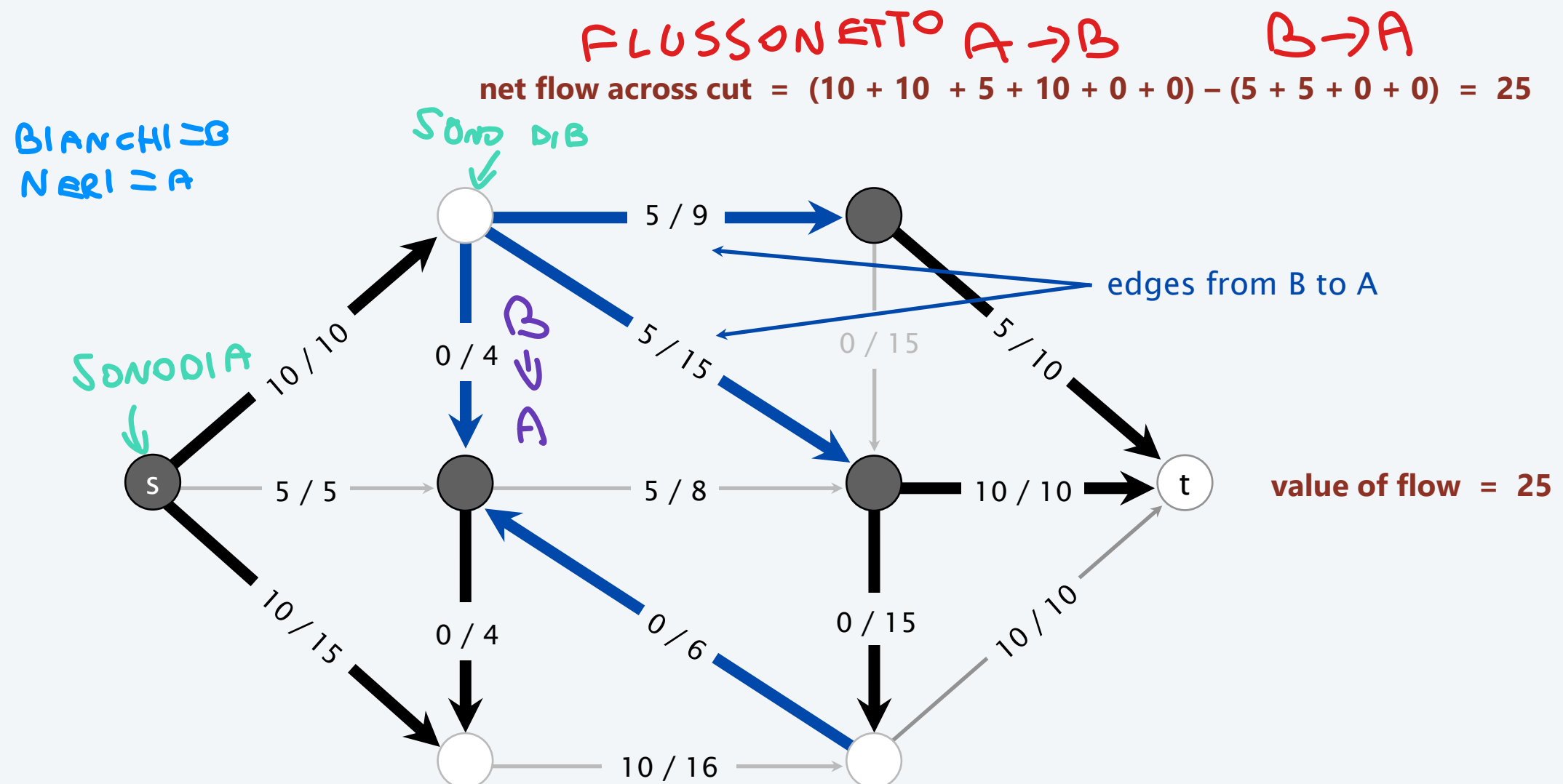
Relationship between flows and cuts

Flow value lemma. Let f be any flow and let (A, B) be any cut. Then, the value of the flow f equals the net flow across the cut (A, B) .

QUALSiasi sTcut cIdA il valore del flow

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$

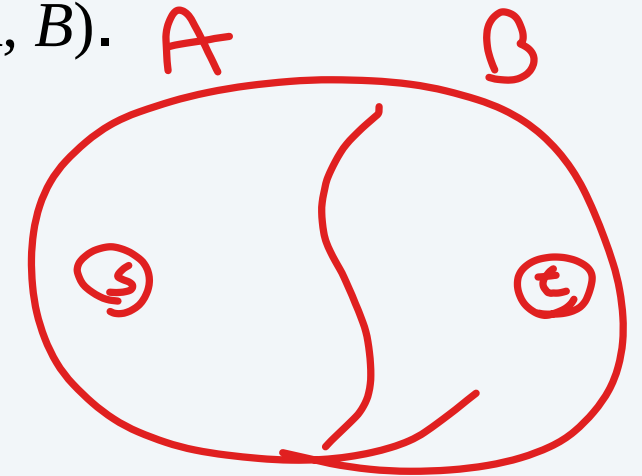
- Sì, per qualsiasi cut (A, B) con $s \in A$ e $t \in B$ (cioè uno s - t cut), il **net flow** attraverso il cut è uguale al valore totale del flusso $val(f)$.
- E in generale, anche per qualsiasi taglio (A, B) (non per forza solo s - t), il **net flow across the cut** è sempre uguale a $val(f)$.



Relationship between flows and cuts

Flow value lemma. Let f be any flow and let (A, B) be any cut. Then, the value of the flow f equals the net flow across the cut (A, B) .

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$



Pf.

$$val(f) = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ in to } s} f(e)$$

by flow conservation, all terms
except for $v = s$ are 0

$$\longrightarrow = \sum_{v \in A} \left(\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ in to } v} f(e) \right)$$

$$= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \quad \square$$

SENON PRENDO

IL NODO S E' F

IL FLOW DI IN SARA' = A OUT

QUINDI = 0

Relationship between flows and cuts

Weak duality. Let f be any flow and (A, B) be any cut. Then, $val(f) \leq cap(A, B)$. CAPACITÀ TAGLIO

Pf.

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$

flow value
lemma

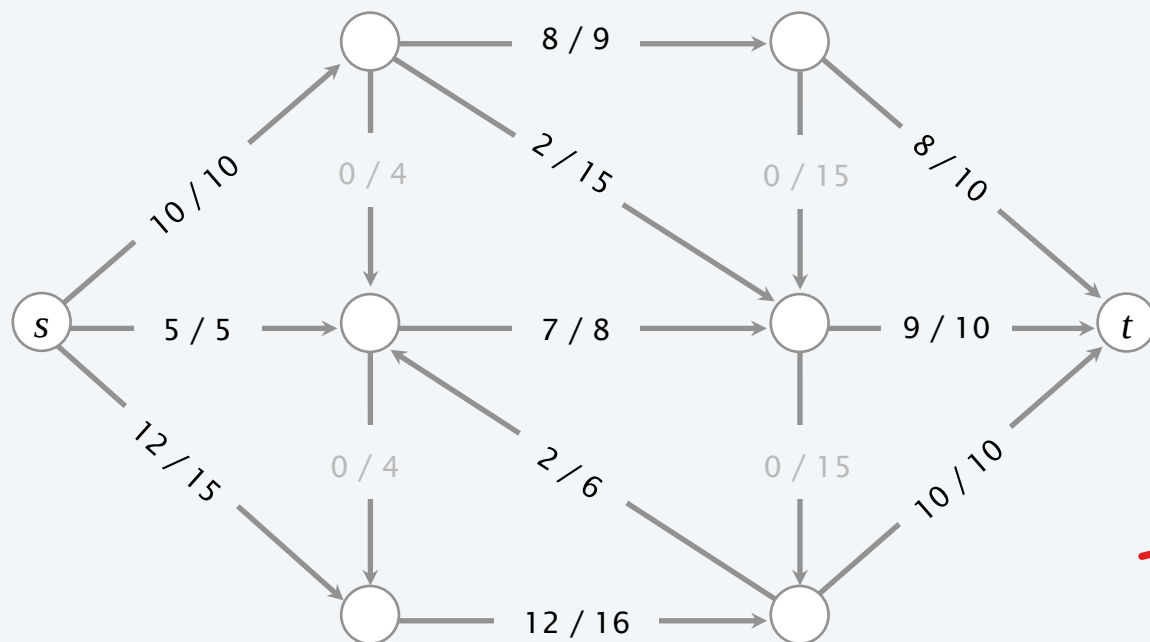
$$\leq \sum_{e \text{ out of } A} f(e)$$

$$\leq \sum_{e \text{ out of } A} c(e)$$

$$= cap(A, B) \quad \blacksquare$$

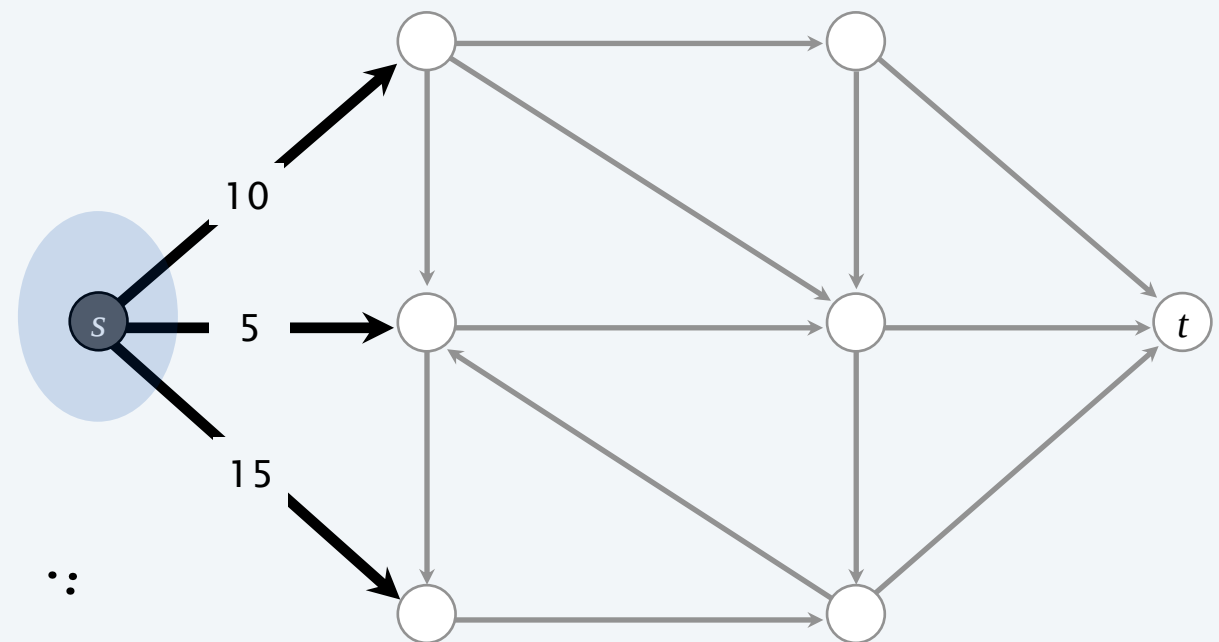
Cio' CHE CONTA E'
OUT OF A PIU' INTO A

- Su un nodo: entrata = uscita.
- Su un insieme di nodi: il flusso netto è **outflow** - inflow, ed è **positivo** (o zero se non c'è produzione).
- La disuguaglianza nasce perché buttiamo via il **termine negativo** per ottenere una **stima superiore**.



value of flow = 27

$\leq$



capacity of cut = 30

Certificate of optimality

Corollary. Let f be a flow and let (A, B) be any cut.

If $val(f) = cap(A, B)$, then f is a max flow and (A, B) is a min cut.

PUNTO DI INCONTRO

Pf.

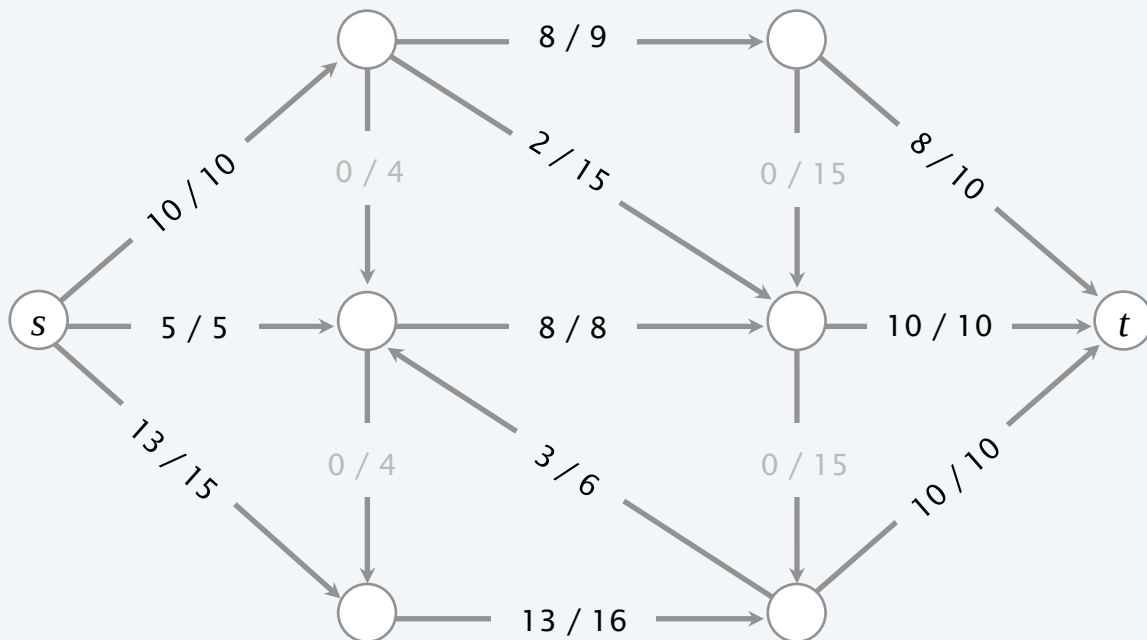
- For any flow f' : $val(f') \leq cap(A, B) = val(f)$.
- For any cut (A', B') : $cap(A', B') \geq val(f) = cap(A, B)$.

weak duality

weak duality

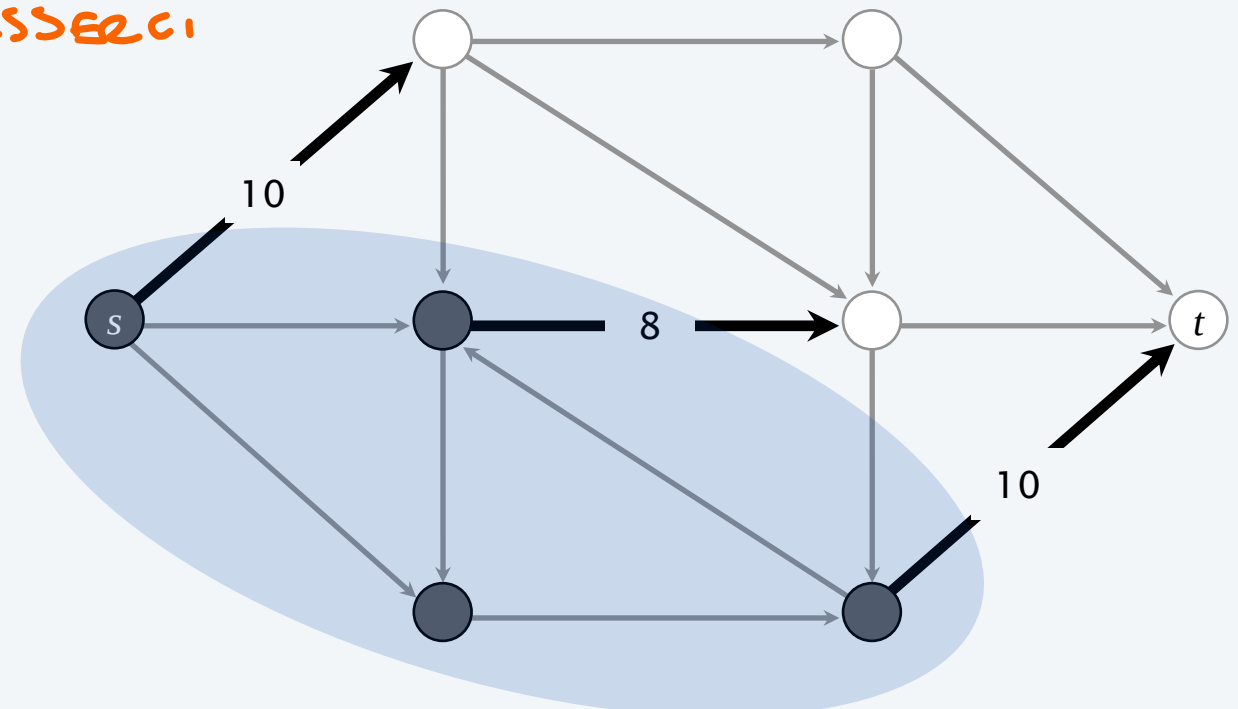
○ CAPTA. FLUSSO MAX
X FLUSSO MIN CUT

⚠ NON STIAMO DICENDO CHE $\exists$ PERFORZA $cap = val$
LO SPIEGHIAMO DOPO $\rightarrow$ BENSÌ POTREBBE ESSERE CI



value of flow = 28

=



capacity of cut = 28

Max-flow min-cut theorem

Max-flow min-cut theorem. Value of a max flow = capacity of a min cut.

strong duality

MAXIMAL FLOW THROUGH A NETWORK

L. R. FORD, JR. AND D. R. FULKERSON

Introduction. The problem discussed in this paper was formulated by T. Harris as follows:

“Consider a rail network connecting two cities by way of a number of intermediate cities, where each link of the network has a number assigned to it representing its capacity. Assuming a steady state condition, find a maximal flow from one given city to the other.”

ON THE MAX FLOW MIN CUT THEOREM OF NETWORKS

G. B. Dantzig
D. R. Fulkerson

P-826 *dy*

April 15, 1955

A Note on the Maximum Flow Through a Network*

P. ELIAS†, A. FEINSTEIN‡, AND C. E. SHANNON§

Summary—This note discusses the problem of maximizing the rate of flow from one terminal to another, through a network which consists of a number of branches, each of which has a limited capacity. The main result is a theorem: The maximum possible flow from left to right through a network is equal to the minimum value among all simple cut-sets. This theorem is applied to solve a more general problem, in which a number of input nodes and a number of output nodes are used.

from one terminal to the other in the original network passes through at least one branch in the cut-set. In the network above, some examples of cut-sets are (d, e, f) , and (b, c, e, g, h) , (d, g, h, i) . By a *simple cut-set* we will mean a cut-set such that if any branch is omitted it is no longer a cut-set. Thus (d, e, f) and (b, c, e, g, h) are simple cut-sets while (d, a, b, c) is not. When a simple cut set is

Max-flow min-cut theorem

Max-flow min-cut theorem. Value of a max flow = capacity of a min cut.

Augmenting path theorem. A flow f is a max flow iff no augmenting paths.

ALGORITMO TERMINA QUANDO ↗

Pf. The following three conditions are equivalent for any flow f :

i. There exists a cut (A, B) such that $cap(A, B) = val(f)$.

ii. f is a max flow.

iii. There is no augmenting path with respect to f .

← if Ford-Fulkerson terminates,
then f is max flow

SONO EQUIVALENTI
OTUTTE VERE OTUTTE FALSE

→ PER DIMOSTRARLO

i → ii → iii

SE UNA È VERA
L'ALTRA È VERA

NELLA RETE RESIDUA

[i ⇒ ii]

ASJUNO CHE i È VERA

▪ This is the weak duality corollary. ▪

$$CAP(A, B) = flow(v)$$

QUELLO DI PRIMA

Max-flow min-cut theorem

Max-flow min-cut theorem. Value of a max flow = capacity of a min cut.

Augmenting path theorem. A flow f is a max flow iff no augmenting paths.

Pf. The following three conditions are equivalent for any flow f :

- i. There exists a cut (A, B) such that $cap(A, B) = val(f)$.
- ii. f is a max flow. ✓ ? SARA'VERA?
- iii. There is no augmenting path with respect to f .

[ii $\Rightarrow$ iii] We prove contrapositive: $\neg$ iii $\Rightarrow$ $\neg$ ii.

- Suppose that there is an augmenting path with respect to f .
- Can improve flow f by sending flow along this path.
- Thus, f is not a max flow. ▪

Max-flow min-cut theorem

ASSUNZIONE VERBA

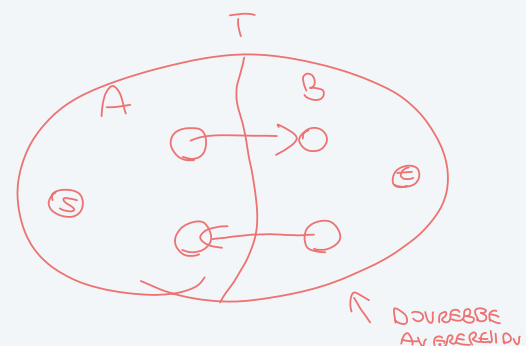
[iii $\Rightarrow$ i]

- Let f be a flow with no augmenting paths.
- Let A = set of nodes reachable from s in residual network G_f .
- By definition of A : $s \in A$.
- By definition of flow f : $t \notin A$.

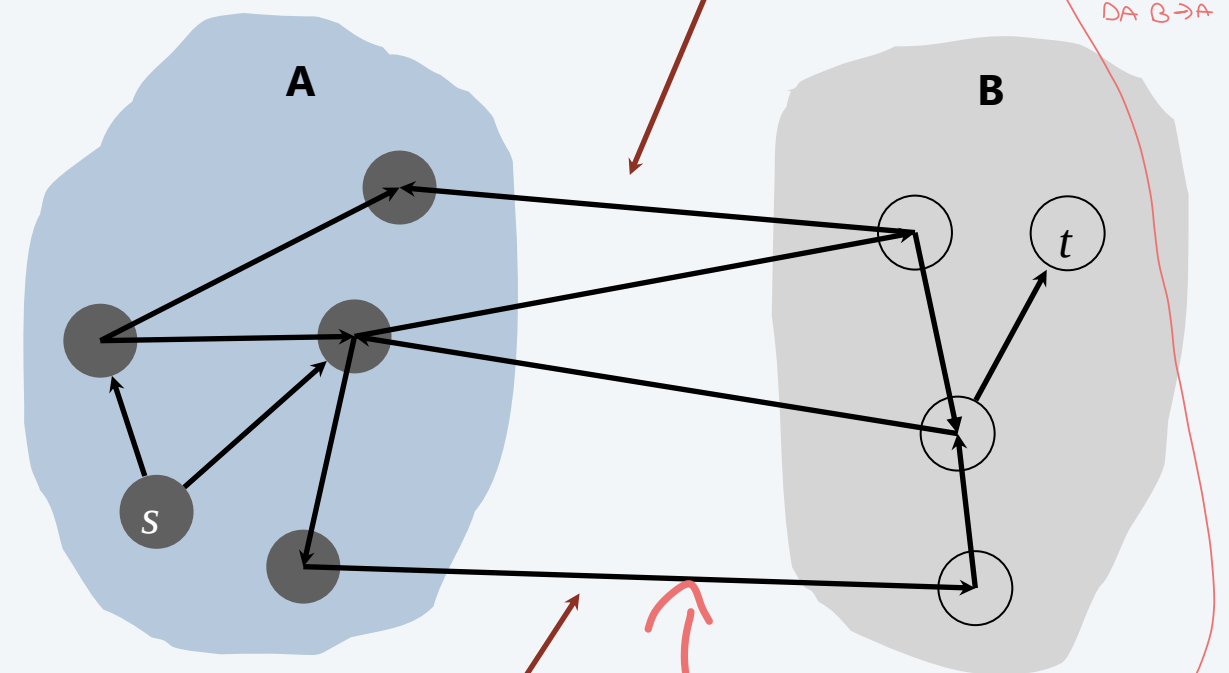
flow value lemma

$$\begin{aligned} \text{val}(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &= \sum_{e \text{ out of } A} c(e) - 0 \\ &= \text{cap}(A, B) \quad \blacksquare \end{aligned}$$

• Partiamo da un flusso massimo f (cioè senza cammini aumentanti).
 • Prendiamo A come i nodi raggiungibili da s nel grafo residuo.
 • Applichiamo il flow value lemma:
 $\text{val}(f) = \sum_{\text{out of } A} f(e) - \sum_{\text{into } A} f$
 • Gli archi da A a B sono saturi ($f(e) = c(e)$).
 • Gli archi da B a A hanno flusso nullo ($f(e) = 0$).
 • Quindi:
 $\text{val}(f) = \text{cap}(A, B)$
 quindi (A, B) è un min cut, e abbiamo dimostrato (iii) $\Rightarrow$ (i).



original flow network G



edge $e = (v, w)$ with $v \in B, w \in A$
must have $f(e) = 0$

edge $e = (v, w)$ with $v \in A, w \in B$
must have $f(e) = c(e)$

SE CI FOSSE
RESIDUO DOVREI AVERE
UN ARCO
DA B -> A

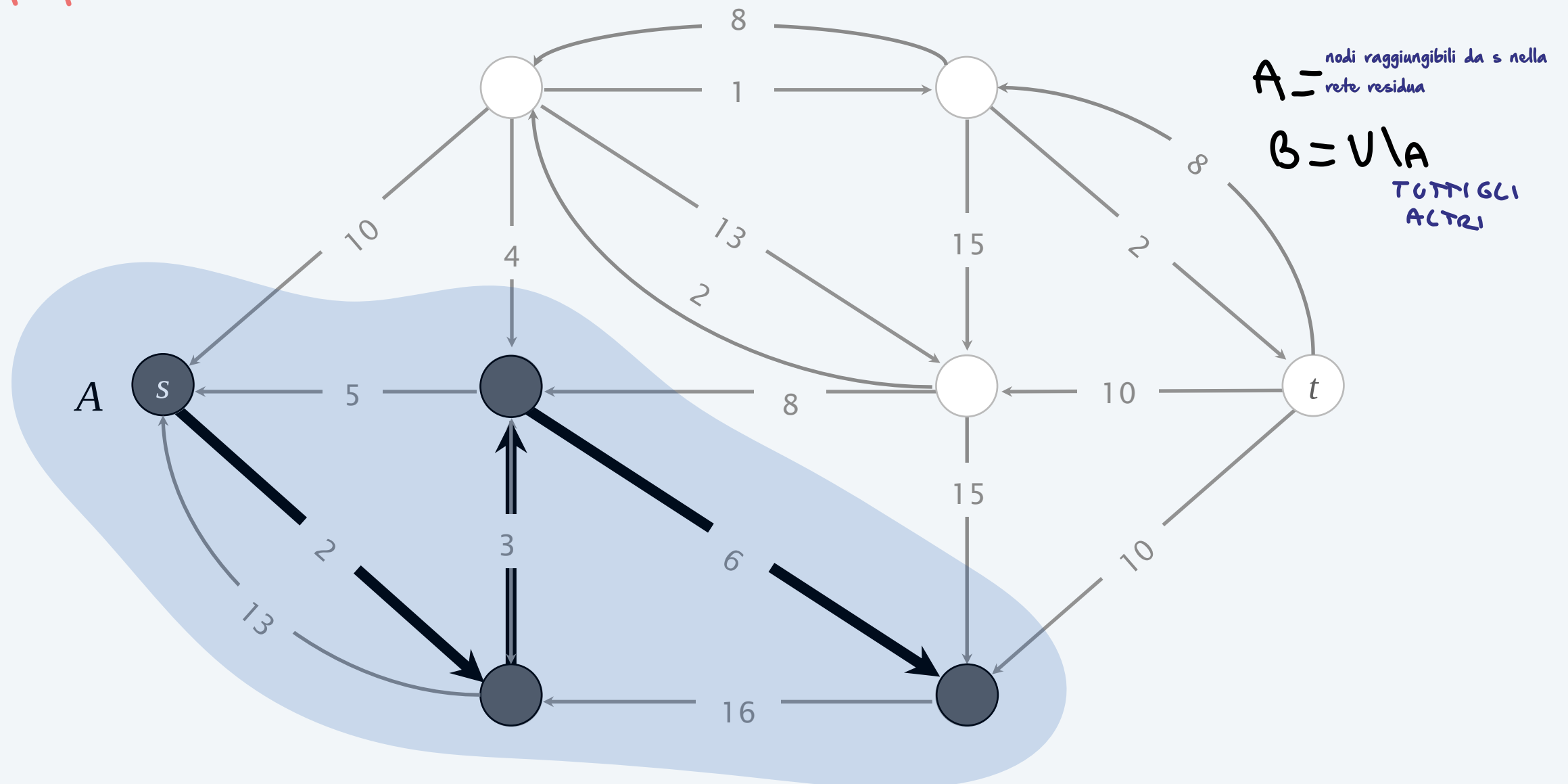
Computing a minimum cut from a maximum flow

Theorem. Given any max flow f , can compute a min cut (A, B) in $O(m)$ time.

Pf. Let A = set of nodes reachable from s in residual network G_f . ■

argument from previous slide implies that
capacity of (A, B) = value of flow f

RISOLVIAMO MAX FLOW E IN LINEARE TROVIAMO MIN CUT



Analysis of Ford–Fulkerson algorithm (for integral capacities)

Assumption. Every edge capacity $c(e)$ is an integer between 1 and C .

SE CAP INTERE FLUSSO $\rightarrow$ INTERO E ANCHE RESIDUI $\rightarrow$ INTERI
se le capacità sono intere allora anche il flusso sarà intero e anche i residui saranno interi

Integrality invariant. Throughout Ford–Fulkerson, every edge flow $f(e)$ and residual capacity $c_f(e)$ is an integer.

Pf. By induction on the number of augmenting paths. ■

*consider cut $A = \{s\}$
 (assumes no parallel edges)*

Theorem. Ford–Fulkerson terminates after at most $val(f^*) \leq nC$ augmenting paths, where f^* is a max flow.

*↑ CAPACITÀ
 #VERTICI*

Pf. Each augmentation increases the value of the flow by at least 1. ■

3. Teorema: Ford-Fulkerson termina
 L'algoritmo termina dopo al più $val(f^*) \leq nC$ aumenti.
Perché?
 • Ogni augmenting path aumenta il flusso di almeno 1 (dato che le capacità sono intere).
 • Il flusso massimo f^* non può superare nC (perché ogni nodo può mandare al massimo flusso pari a C , e ci sono al massimo n nodi in uscita dal taglio).

Corollary. The running time of Ford–Fulkerson is $O(m \cdot val(f^*)) = O(mnC)$.

*$m = \text{ARCHI}$
 $n = \text{VERTICI}$
 $C = \text{CAPACITÀ}$
 PSEUDO POLINOMIALE*

Pf. Can use either BFS or DFS to find an augmenting path in $O(m)$ time.

$f(e)$ is an integer for every e

Integrality theorem. There exists an integral max flow f^* .

Pf. Since Ford–Fulkerson terminates, theorem follows from integrality invariant (and augmenting path theorem). ■

4. Corollario: tempo di esecuzione
 Tempo totale:
 $O(m \cdot val(f^*)) = O(mnC)$
Perché?
 • Ogni iterazione cerca un cammino aumentante (costo $O(m)$).
 • Al massimo $val(f^*) \leq nC$ iterazioni.
 Se scegli il cammino aumentante con **BFS** (come fa Edmonds-Karp), ottieni un comportamento più prevedibile.

5. Integrality theorem
 Esiste sempre un flusso massimo intero.
 Questa è una proprietà bellissima del problema di massimo flusso:
 • Se tutte le capacità sono intere,
 • Allora esiste un flusso massimo intero.
 • E Ford-Fulkerson lo trova!

Ford–Fulkerson: exponential example *ESFMP10*

Q. Is generic Ford–Fulkerson algorithm poly-time in input size?

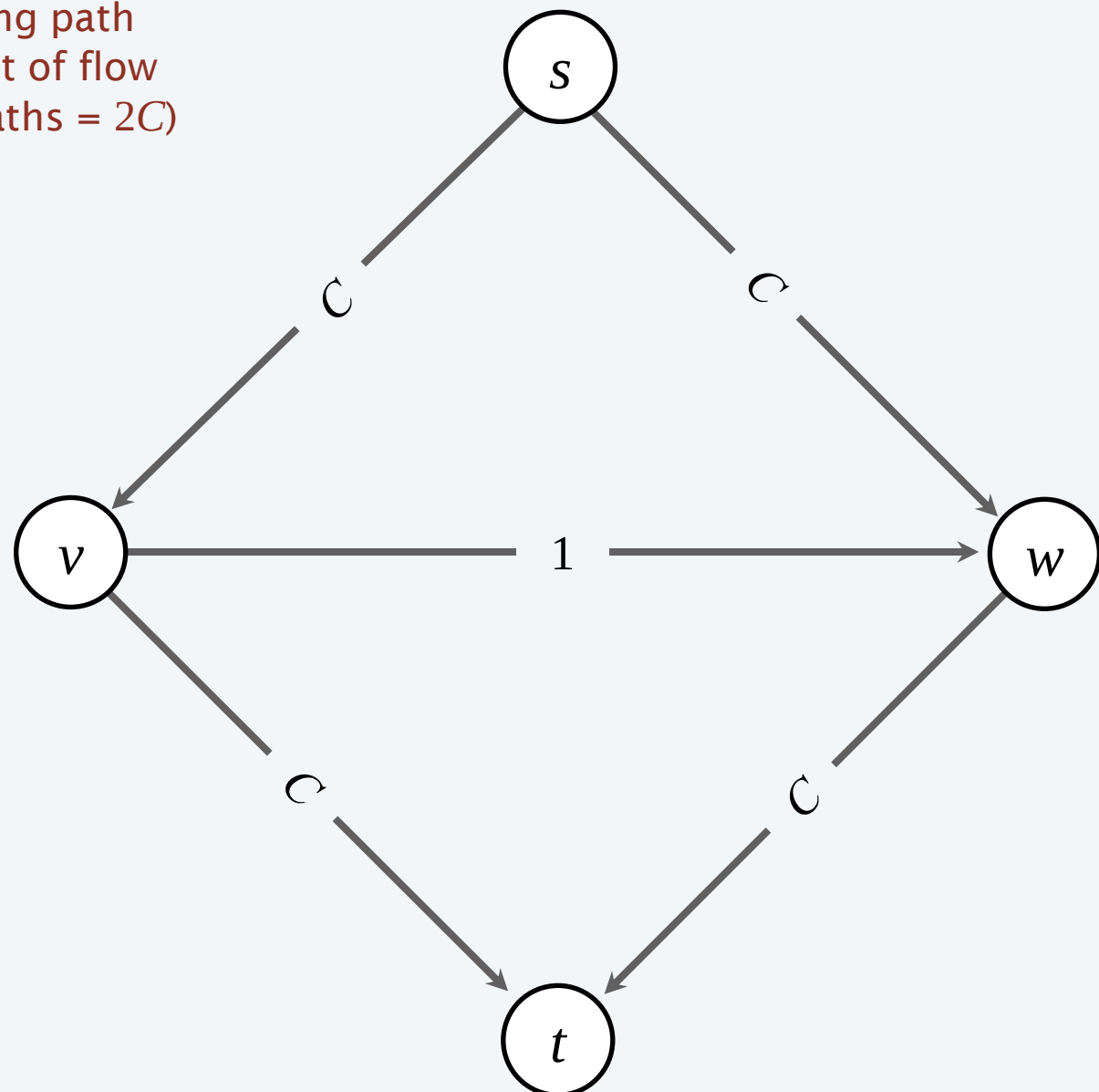
$m, n,$ and $\log C$

A. No. It is pseudo-polynomial.

If max capacity is C , then algorithm can take $\geq C$ iterations.

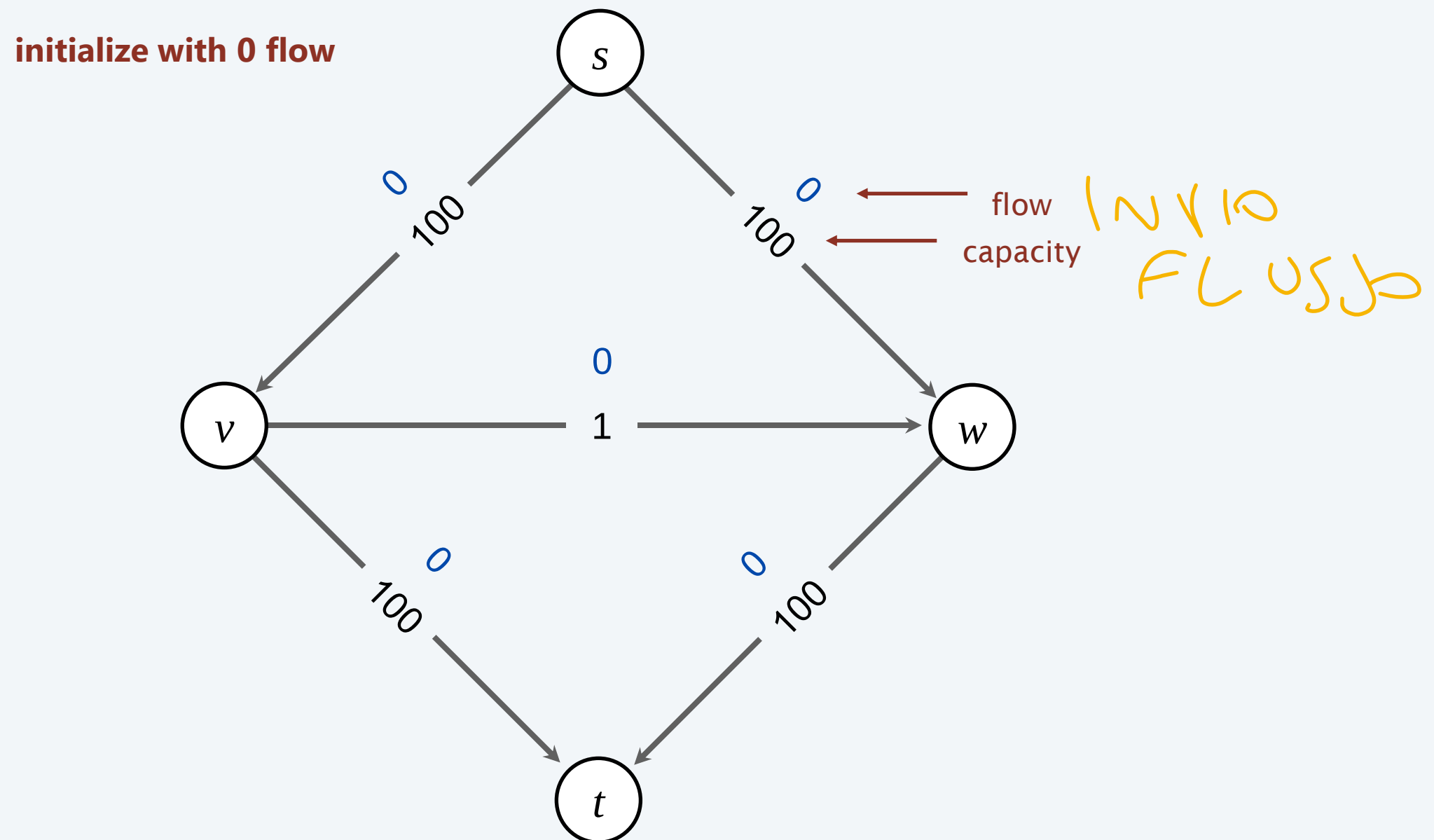
- $s \rightarrow v \rightarrow w \rightarrow t$
- $s \rightarrow w \rightarrow v \rightarrow t$
- $s \rightarrow v \rightarrow w \rightarrow t$
- $s \rightarrow w \rightarrow v \rightarrow t$
- ...
- $s \rightarrow v \rightarrow w \rightarrow t$
- $s \rightarrow w \rightarrow v \rightarrow t$

each augmenting path
sends only 1 unit of flow
(# augmenting paths = $2C$)



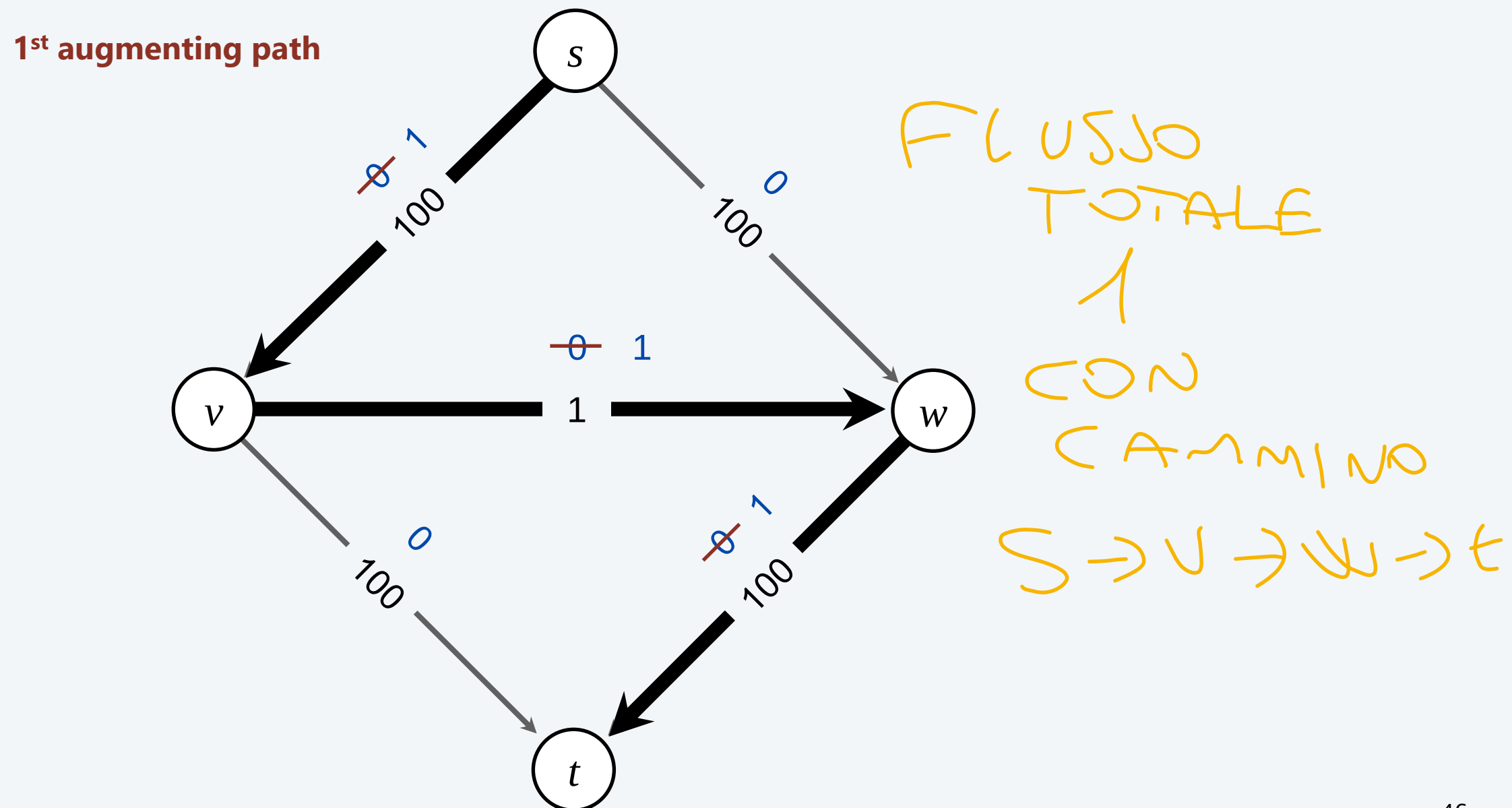
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



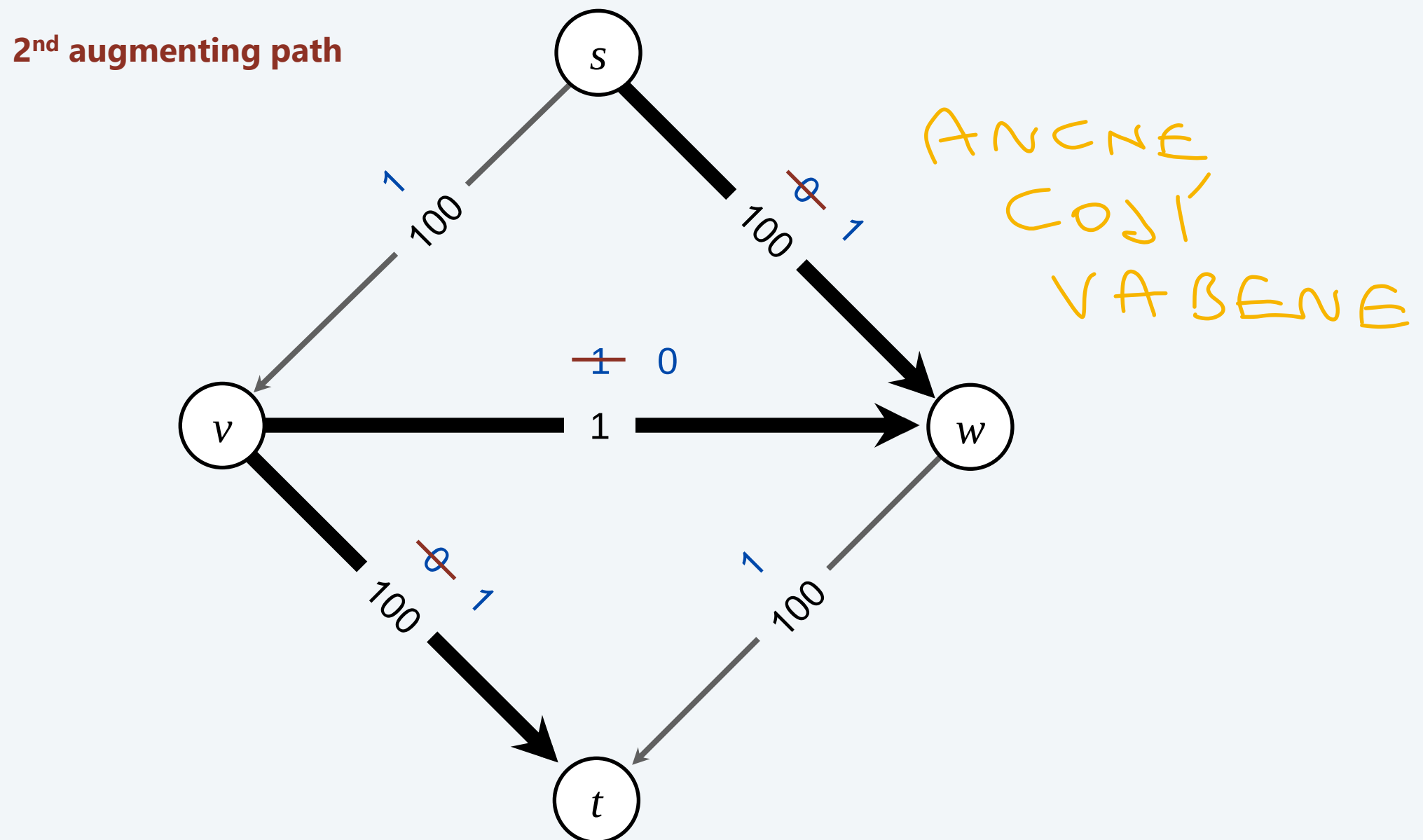
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



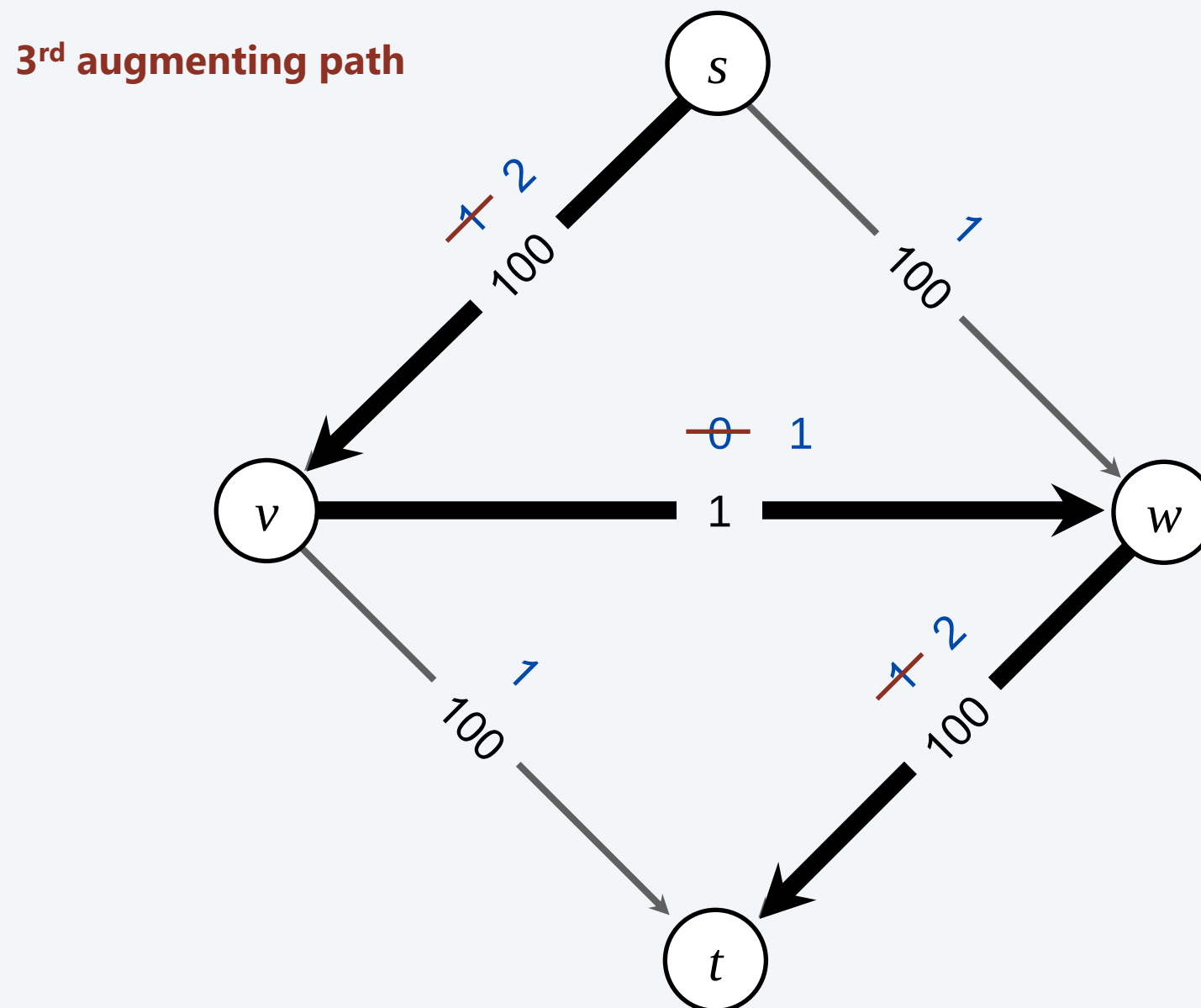
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



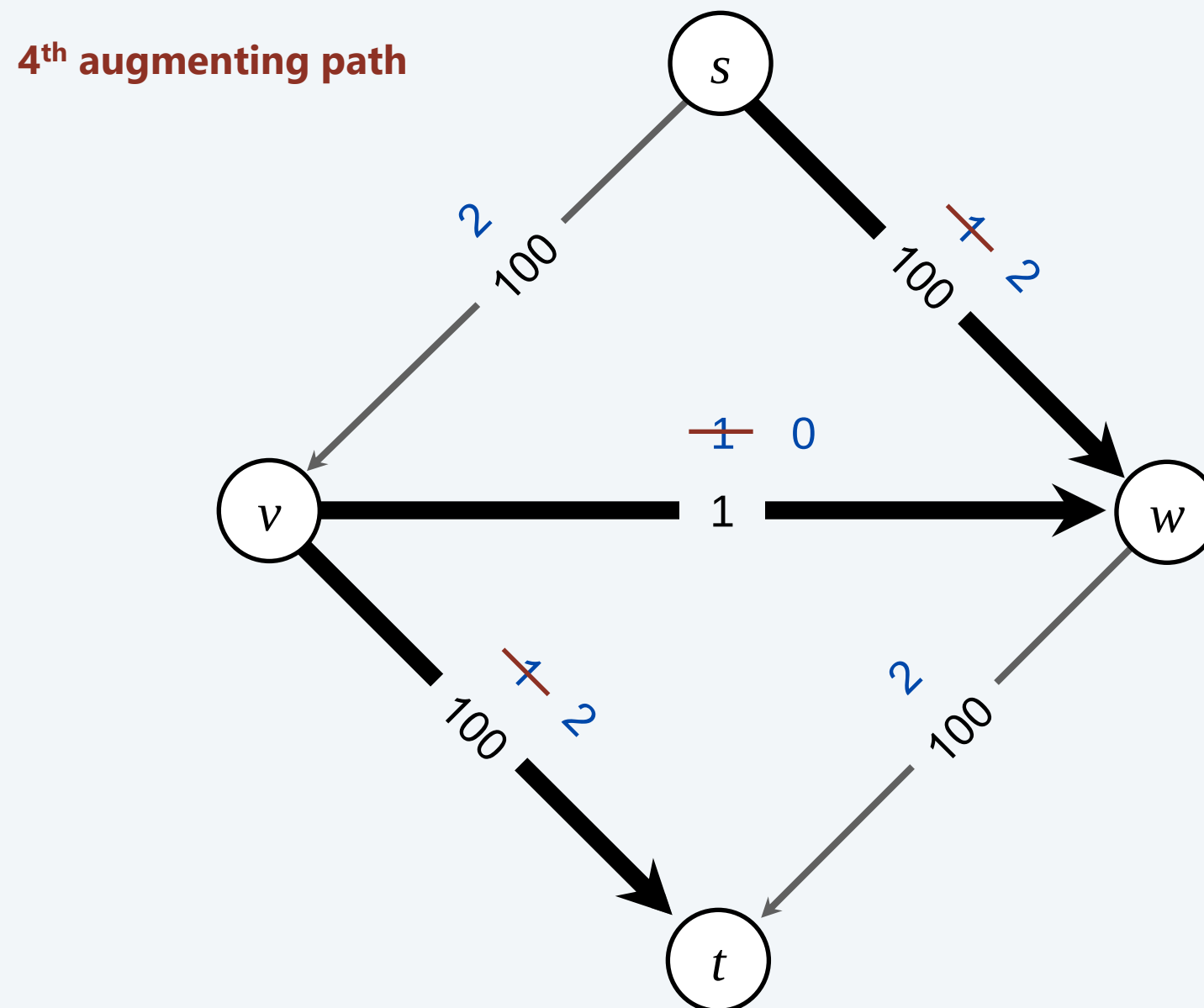
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



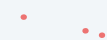
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



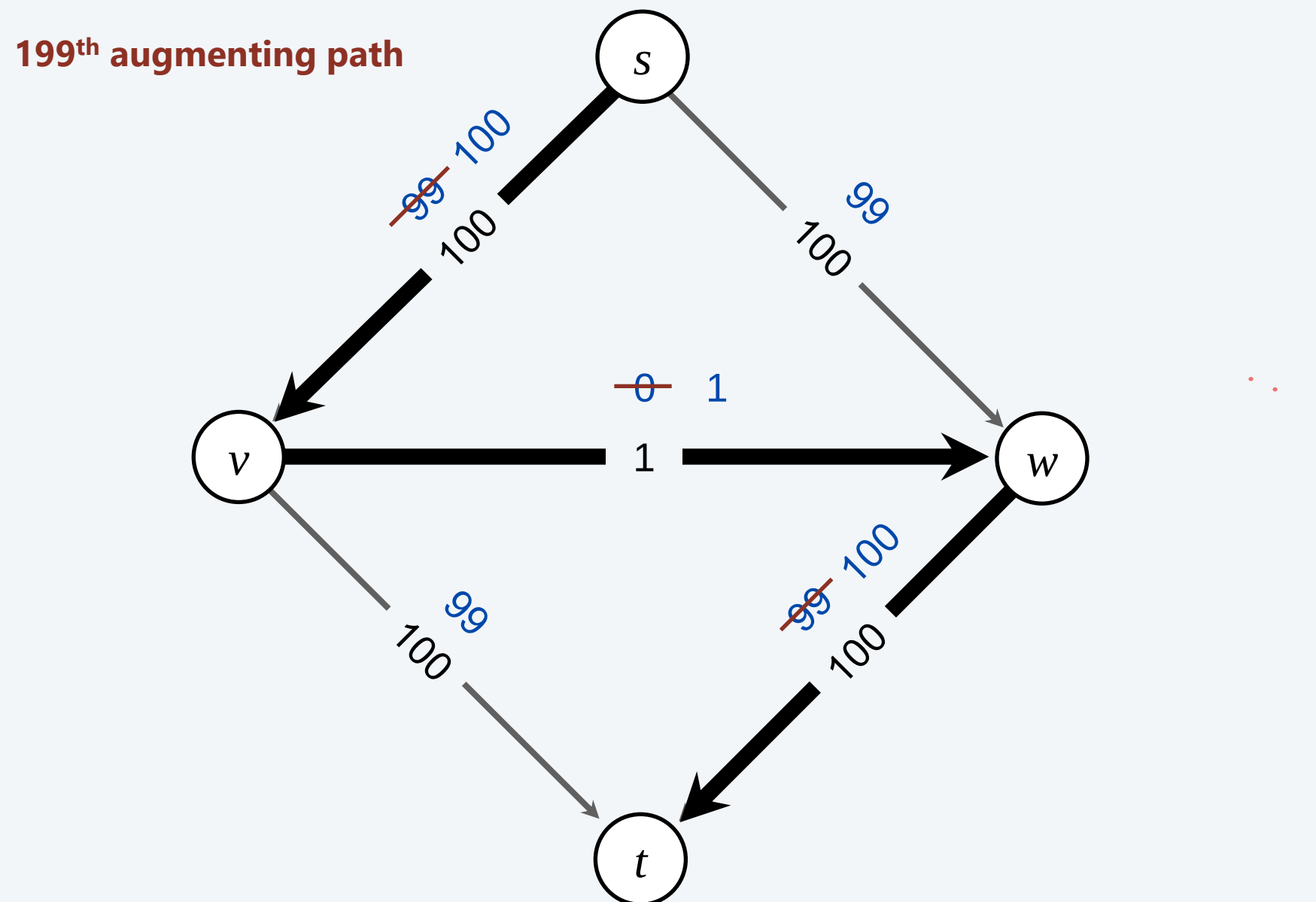
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



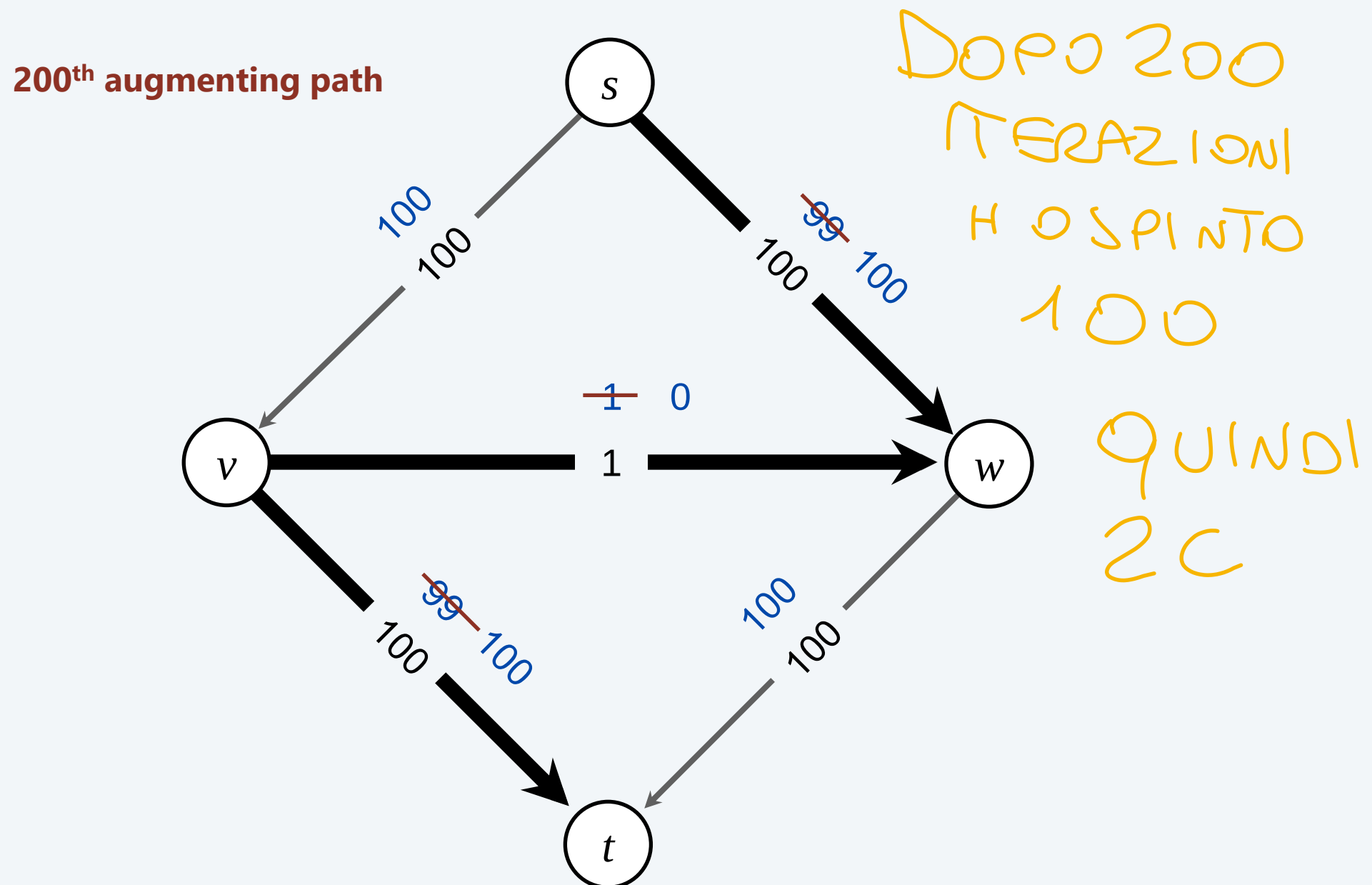
Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.



Ford–Fulkerson algorithm: exponential-time example

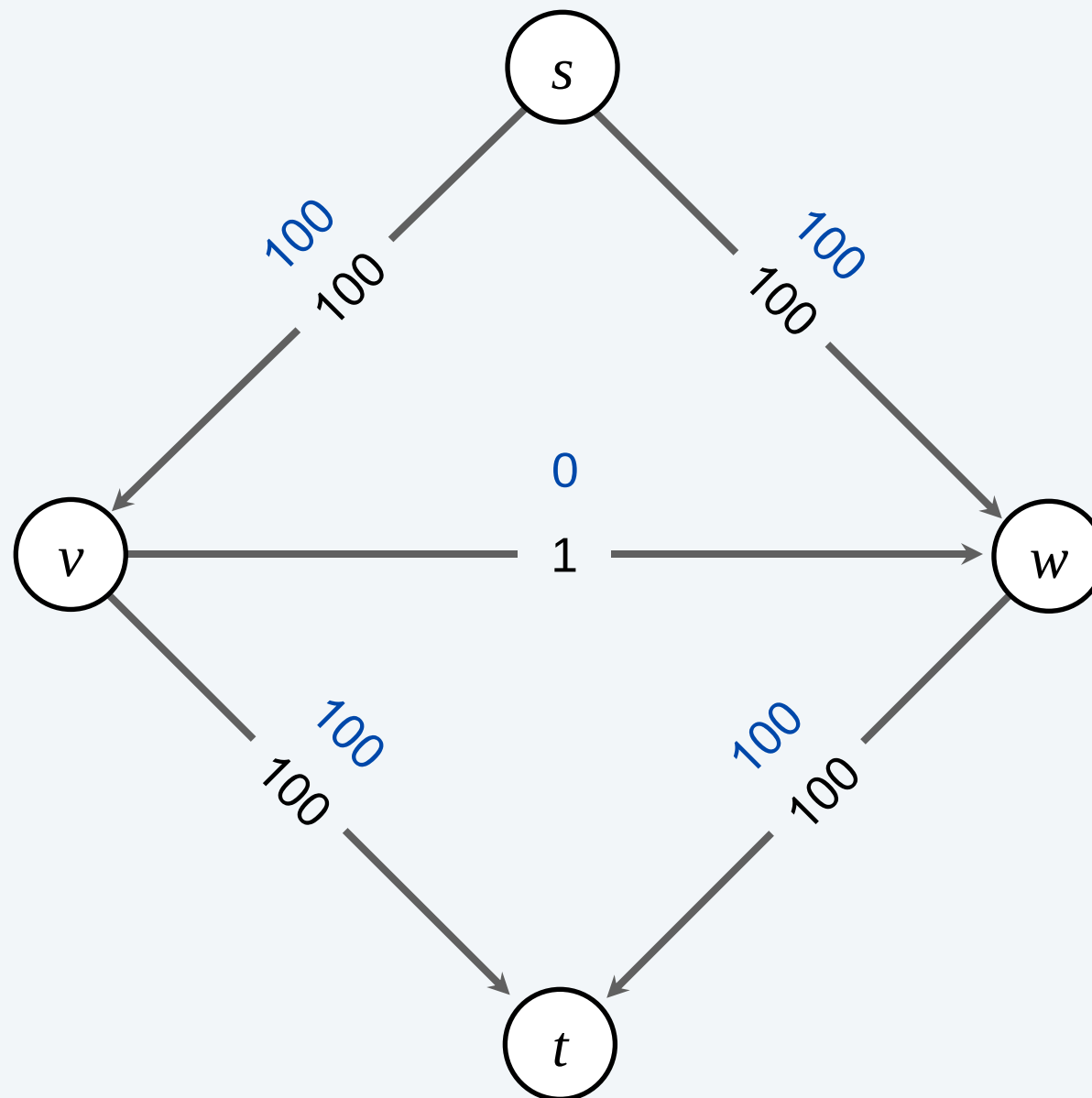
Bad news. Number of augmenting paths can be exponential in input size.

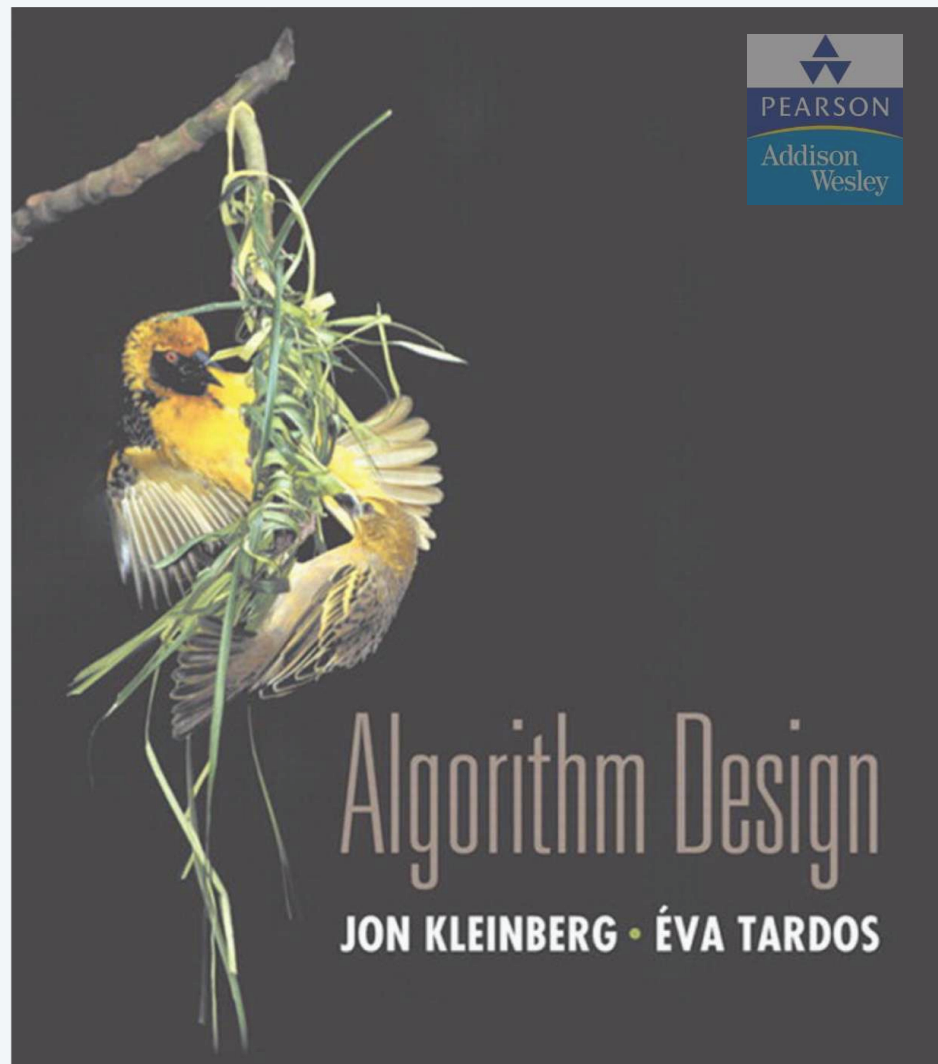


Ford–Fulkerson algorithm: exponential-time example

Bad news. Number of augmenting paths can be exponential in input size.

*SE C E' PICCOLO ALG BUONO! C = 1 milione
⇒ 2 milioni*





SECTION 7.3

7. NETWORK FLOW I

- *max-flow and min-cut problems*
- *Ford–Fulkerson algorithm*
- *max-flow min-cut theorem*
- *choosing good augmenting paths*

Choosing good augmenting paths ← MIGLIORIAMO

Use care when selecting augmenting paths.

- Some choices lead to exponential algorithms.
 - Clever choices lead to polynomial algorithms.
- ← BISOGNA SCEGLIERE BENE

Pathology. When edge capacities can be irrational, no guarantee that Ford–Fulkerson terminates (or converges to a maximum flow)!

NO IRRAZIONALI'.

Goal. Choose augmenting paths so that:

- Can find augmenting paths efficiently. $O(m)$
- Few iterations. POCO AUMENTI

2 MODI



Choosing good augmenting paths

Choose augmenting paths with:

- Sufficiently large bottleneck capacity. *MOD01 PONGOUNA BOT CAP ALTA*
- Fewest edges. *MOD02*
NUMERO MINI MODI ARCHI

Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems

JACK EDMONDS

University of Waterloo, Waterloo, Ontario, Canada

AND

RICHARD M. KARP

University of California, Berkeley, California

ABSTRACT. This paper presents new algorithms for the maximum flow problem, the Hitchcock transportation problem, and the general minimum-cost flow problem. Upper bounds on the numbers of steps in these algorithms are derived, and are shown to compare favorably with upper bounds on the numbers of steps required by earlier algorithms.

Edmonds-Karp 1972 (USA)

Capacity-scaling algorithm

1. **Overview.** Choosing augmenting paths with “large” bottleneck capacity.

- Maintain scaling parameter Δ .
- Let $G_f(\Delta)$ be the part of the residual network containing only those edges with capacity $\geq \Delta$.
- Any augmenting path in $G_f(\Delta)$ has bottleneck capacity $\geq \Delta$.

though not necessarily largest

GUARDO SOLO ARCHI CON ALMENO Δ

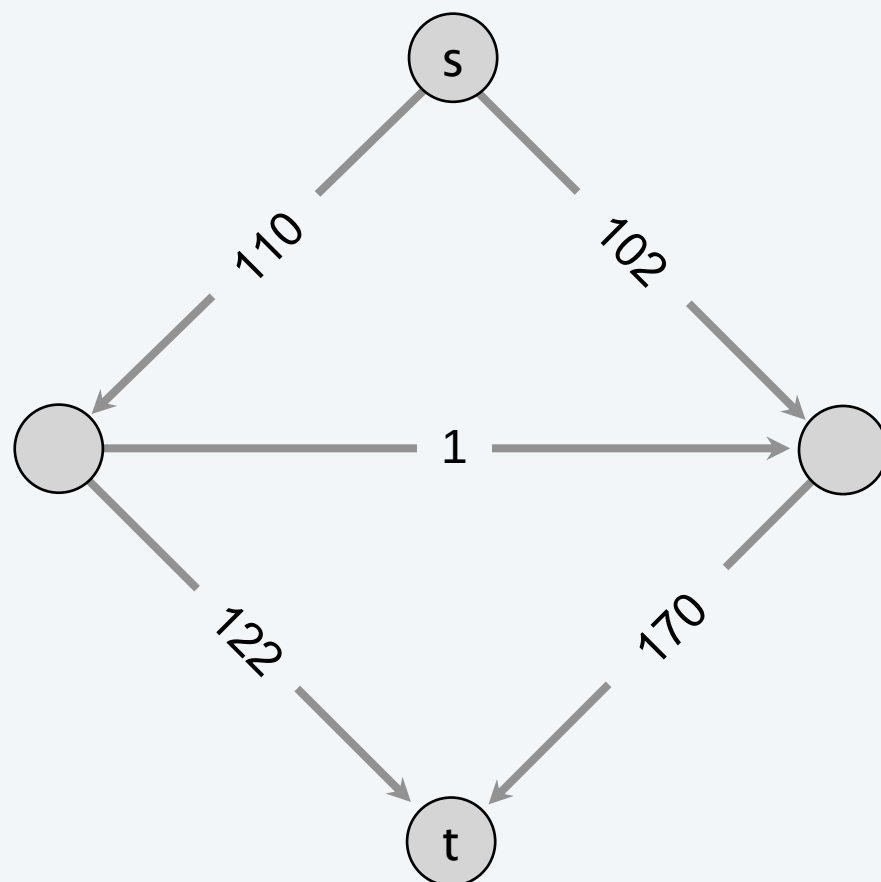
Bottleneck capacity: significato semplice

Bottleneck capacity di un cammino aumentante =

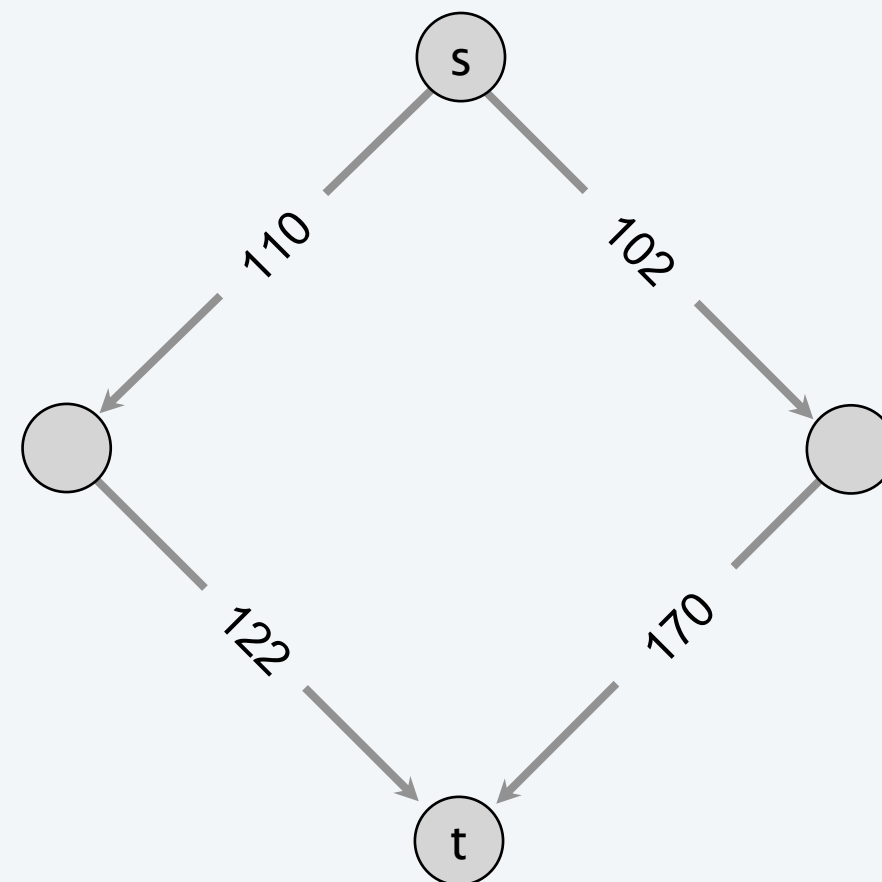
La capacità più piccola tra tutti gli archi del cammino.

In altre parole:

- Se hai un percorso da s a t ,
- Guardi tutti gli archi lungo il percorso,
- Prendi il minimo delle capacità residue di quegli archi: quello è il **bottleneck**.



G_f



$G_f(\Delta), \Delta = 100$

Capacity-scaling algorithm

CAPACITY-SCALING(G)

FOREACH edge $e \in E$: $f(e) \leftarrow 0$.

$\Delta \leftarrow$ largest power of 2 $\leq C$.

POTENZA DI
2

WHILE ($\Delta \geq 1$)

$G_f(\Delta) \leftarrow \Delta$ -residual network of G with respect to flow f .

WHILE (there exists an $s \rightsquigarrow t$ path P in $G_f(\Delta)$)

$f \leftarrow$ AUGMENT(f, c, P).

Update $G_f(\Delta)$.

$\Delta \leftarrow \Delta / 2$. $\leftarrow$ DIMINUIRE Δ FINO A $\Delta=1$

Δ -scaling phase

RETURN f .

Capacity-scaling algorithm: analysis of running time (sketch)

It can be proved the following:

Lemma 1. There are $1 + \lfloor \log_2 C \rfloor$ scaling phases.

LAUORIAMO IN POW DI 2

RICORDA SOLO A MEMORIA NO DIM

Lemma 2. There are $\leq 2m$ augmentations per scaling phase.

➡ total number of augmentations: $O(m \log C)$

Theorem. The capacity-scaling algorithm takes $O(m^2 \log C)$ time.

Shortest augmenting path *MODO 2*

Q. How to choose next augmenting path in Ford-Fulkerson?

A. Pick one that uses the fewest edges.

*SCELGO
CAMMINO PIU' CORTO*

 can find via BFS

SHORTEST-AUGMENTING-PATH(G)

FOREACH $e \in E$: $f(e) \leftarrow 0$.

$G_f \leftarrow$ residual network of G with respect to flow f .

WHILE (there exists an $s \rightsquigarrow t$ path in G_f)

$P \leftarrow$ **BREADTH-FIRST-SEARCH**(G_f).

$f \leftarrow$ **AUGMENT**(f, c, P).

Update G_f .

RETURN f .

Shortest augmenting path: running time

It can be proved the following:

Lemma 1. The total number of augmentations is at most mn .

Theorem. The shortest-augmenting-path algorithm takes $O(m^2 n)$ time.

● Perché il numero di augmentations è $\leq mn$?

L'idea è questa:

- Ogni volta che fai un'augmentazione (cioè mandi un po' di flusso lungo un cammino più corto),
- Almeno un arco "critico" (cioè saturo o liberato) cambia.
- E la distanza da s a t nella rete residua aumenta.
- Non puoi aumentare la distanza troppe volte, perché la distanza massima è n (al massimo attraversi tutti i nodi).
- Quindi ogni arco può diventare saturo solo un numero limitato di volte.

In totale:

- m archi $\times n$ livelli di distanza $\Rightarrow O(mn)$ augmentations.

● Theorem

L'algoritmo del cammino aumentante più corto richiede $O(m^2 n)$ tempo.

Perché?

- Ogni augmentazione richiede un BFS per trovare il cammino più corto $\rightarrow$ costo $O(m)$ (per BFS su m archi).
- E fai al massimo $O(mn)$ augmentations.

Quindi:

$$O(m) \times O(mn) = O(m^2 n)$$

Augmenting-path algorithms: summary

year	method	# augmentations	running time	
1955	augmenting path	$n C$	$O(m n C)$	
1972	fattest path	$m \log (mC)$	$O(m^2 \log n \log (mC))$	fat paths
1972	capacity scaling	$m \log C$	$O(m^2 \log C)$	
1985	improved capacity scaling	$m \log C$	$O(m n \log C)$	
1970	shortest augmenting path	$m n$	$O(m^2 n)$	shortest paths
1970	level graph	$m n$	$O(m n^2)$	
1983	dynamic trees	$m n$	$O(m n \log n)$	

augmenting-path algorithms with m edges, n nodes, and integer capacities between 1 and C

Maximum-flow algorithms: theory highlights

year	method	worst case	discovered by
1951	simplex	$O(m n^2 C)$	Dantzig
1955	augmenting paths	$O(m n C)$	Ford–Fulkerson
1970	shortest augmenting paths	$O(m n^2)$	Edmonds–Karp, Dinitz
1974	blocking flows	$O(n^3)$	Karzanov
1983	dynamic trees	$O(m n \log n)$	Sleator–Tarjan
1985	improved capacity scaling	$O(m n \log C)$	Gabow
1988	push–relabel	$O(m n \log (n^2 / m))$	Goldberg–Tarjan
1998	binary blocking flows	$O(m^{3/2} \log (n^2 / m) \log C)$	Goldberg–Rao
2013	compact networks	$O(m n)$	Orlin
2014	interior-point methods	$\tilde{O}(m n^{1/2} \log C)$	Lee–Sidford
2016	electrical flows	$\tilde{O}(m^{10/7} C^{1/7})$	Mądry
20xx			

max-flow algorithms with m edges, n nodes, and integer capacities between 1 and C

Maximum Flow and Minimum-Cost Flow in Almost-Linear Time

Li Chen*
Georgia Tech
lichen@gatech.edu

Rasmus Kyng†
ETH Zurich
kyng@inf.ethz.ch

Yang P. Liu‡
Stanford University
yangpliu@stanford.edu

Richard Peng
University of Waterloo §
y5peng@uwaterloo.ca

Maximilian Probst Gutenberg†
ETH Zurich
maxprobst@ethz.ch

Sushant Sachdeva¶
University of Toronto
sachdeva@cs.toronto.edu

April 26, 2022

Abstract

We give an algorithm that computes exact maximum flows and minimum-cost flows on directed graphs with m edges and polynomially bounded integral demands, costs, and capacities in $m^{1+o(1)}$ time. Our algorithm builds the flow through a sequence of $m^{1+o(1)}$ approximate undirected minimum-ratio cycles, each of which is computed and processed in amortized $m^{o(1)}$ time using a new dynamic graph data structure.

Our framework extends to algorithms running in $m^{1+o(1)}$ time for computing flows that minimize general edge-separable convex functions to high accuracy. This gives almost-linear time algorithms for several problems including entropy-regularized optimal transport, matrix scaling, p -norm flows, and p -norm isotonic regression on arbitrary directed acyclic graphs.